

方块解压馆软件 V1.0.0 源代码

```
namespace Model.UI
{
    public enum BlockRelaxItemType
    {
        None = 0,
        JellyBlue = 1,
        JellyRed = 2,
        JellyYellow = 3,
        JellyGreen = 4,
        JellyPurple = 5,
        JellyOrange = 6,
    }
}

namespace Model.UI
{
    public enum BlockRelaxCollectType
    {
        None = 0,
        Collect01 = 1,
        Collect02 = 2,
        Collect03 = 3,
        Collect04 = 4,
        Collect05 = 5,
        Collect06 = 6,
    }
}

using System;
using System.Collections.Generic;
using Model.UI;
using UnityEngine;

public static class BlockRelaxPieceShape
{
    private static readonly List<Vector2Int> TryGetSizeScratchCells = new List<Vector2Int>(16);

    public static bool TryGetSize(Transform pieceRoot, BlockRelaxSelectPiecePool pool, out int width, out int height) =>
    {
        TryGetSize(pieceRoot != null ? pieceRoot.name : null, pool, out width, out height);
        public static bool TryGetSize(string pieceRootName, BlockRelaxSelectPiecePool pool, out int width, out int height)
        {
            width = height = 0;
            var tmp = TryGetSizeScratchCells;
            tmp.Clear();
            if (!TryGetShapeCells(pieceRootName, pool, tmp, out width, out height))
                return false;
            return width > 0 && height > 0;
        }

        public static bool TryGetShapeCells(Transform pieceRoot, BlockRelaxSelectPiecePool pool, List<Vector2Int> into) =>
        {
            TryGetShapeCells(pieceRoot != null ? pieceRoot.name : null, pool, into, out _, out _);
            public static bool TryGetShapeCells(string pieceRootName, BlockRelaxSelectPiecePool pool, List<Vector2Int> into, out int bboxWidth, out int bboxHeight)
            {
                bboxWidth = bboxHeight = 0;
            }
        }
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        if (into == null || string.IsNullOrEmpty(pieceRootName) || pool == null)
            return false;
        const string clone = " (Clone)";
        if (pieceRootName.EndsWith(clone))
            pieceRootName = pieceRootName[..^clone.Length];
        TryStripSelectPieceSpawnNamePrefix(ref pieceRootName);
        if (!pool.TryGetBoardFootprintCells(pieceRootName, into))
            return false;
        ComputeBBox(into, out bboxWidth, out bboxHeight);
        return into.Count > 0;
    }
    private static void TryStripSelectPieceSpawnNamePrefix(ref string pieceRootName)
    {
        const string prefix = "SelectPiece_";
        if (string.IsNullOrEmpty(pieceRootName) || !pieceRootName.StartsWith(prefix,
StringComparison.Ordinal))
            return;
        int i = prefix.Length;
        while (i < pieceRootName.Length && char.IsDigit(pieceRootName[i]))
            i++;
        if (i >= pieceRootName.Length || pieceRootName[i] != '_' )
            return;
        pieceRootName = pieceRootName[(i + 1)..];
    }
    private static void ComputeBBox(IReadOnlyList<Vector2Int> cells, out int bboxWidth, out int
bboxHeight)
    {
        bboxWidth = bboxHeight = 0;
        if (cells == null || cells.Count == 0)
            return;
        int maxX = int.MinValue, maxY = int.MinValue;
        for (int i = 0; i < cells.Count; i++)
        {
            var c = cells[i];
            if (c.x > maxX) maxX = c.x;
            if (c.y > maxY) maxY = c.y;
        }
        bboxWidth = maxX + 1;
        bboxHeight = maxY + 1;
    }
    public static void GetOccupiedCells(int anchorCol, int anchorRow, IReadOnlyList<Vector2Int>
shapeCells, List<Vector2Int> into)
    {
        into.Clear();
        if (shapeCells == null)
            return;
        for (int i = 0; i < shapeCells.Count; i++)
        {
            var o = shapeCells[i];
            into.Add(new Vector2Int(anchorCol + o.x, anchorRow + o.y));
        }
    }
    public static void GetOccupiedCells(int anchorCol, int anchorRow, int width, int height,
```

方块解压馆软件 V1.0.0 源代码

```
List<Vector2Int> into)
{
    into.Clear();
    for (int dy = 0; dy < height; dy++)
    {
        for (int dx = 0; dx < width; dx++)
            into.Add(new Vector2Int(anchorCol + dx, anchorRow + dy));
    }
}

public static bool IsWithinBoard(int anchorCol, int anchorRow, int width, int height, int
boardCols, int boardRows)
{
    if (anchorCol < 0 || anchorRow < 0)
        return false;
    if (anchorCol + width > boardCols || anchorRow + height > boardRows)
        return false;
    return true;
}

public static bool IsWithinBoard(int anchorCol, int anchorRow, IReadOnlyList<Vector2Int>
shapeCells, int boardCols, int boardRows)
{
    if (shapeCells == null)
        return false;
    for (int i = 0; i < shapeCells.Count; i++)
    {
        var t = shapeCells[i];
        int x = anchorCol + t.x;
        int y = anchorRow + t.y;
        if (x < 0 || y < 0 || x >= boardCols || y >= boardRows)
            return false;
    }
    return true;
}

public static bool CanPlace(bool[,] occupied, int boardCols, int boardRows, int anchorCol,
int anchorRow, IReadOnlyList<Vector2Int> shapeCells)
{
    if (shapeCells == null || shapeCells.Count == 0)
        return false;
    if (!IsWithinBoard(anchorCol, anchorRow, shapeCells, boardCols, boardRows))
        return false;
    for (int i = 0; i < shapeCells.Count; i++)
    {
        var o = shapeCells[i];
        if (occupied[anchorCol + o.x, anchorRow + o.y])
            return false;
    }
    return true;
}

public static bool CanPlace(bool[,] occupied, int boardCols, int boardRows, int anchorCol,
int anchorRow, int width, int height)
{
    if (!IsWithinBoard(anchorCol, anchorRow, width, height, boardCols, boardRows))
        return false;
}
```

方块解压馆软件 V1.0.0 源代码

```
        for (int dy = 0; dy < height; dy++)
        {
            for (int dx = 0; dx < width; dx++)
            {
                if (occupied[anchorCol + dx, anchorRow + dy])
                    return false;
            }
        }
        return true;
    }

    public static bool HasAnyPlacementForShape(
        bool[,] occupied,
        int boardCols,
        int boardRows,
        IReadOnlyList<Vector2Int> shapeCells)
    {
        if (occupied == null || shapeCells == null || shapeCells.Count == 0)
            return false;
        if (occupied.GetLength(0) < boardCols || occupied.GetLength(1) < boardRows)
            return false;
        for (int anchorRow = 0; anchorRow < boardRows; anchorRow++)
        {
            for (int anchorCol = 0; anchorCol < boardCols; anchorCol++)
            {
                if (CanPlace(occupied, boardCols, boardRows, anchorCol, anchorRow, shapeCells))
                    return true;
            }
        }
        return false;
    }

    public static void ApplyPlace(bool[,] occupied, int anchorCol, int anchorRow,
        IReadOnlyList<Vector2Int> shapeCells, bool filled = true)
    {
        if (shapeCells == null)
            return;
        for (int i = 0; i < shapeCells.Count; i++)
        {
            var o = shapeCells[i];
            occupied[anchorCol + o.x, anchorRow + o.y] = filled;
        }
    }

    public static void ApplyPlace(bool[,] occupied, int anchorCol, int anchorRow, int width, int
height, bool filled = true)
    {
        for (int dy = 0; dy < height; dy++)
        {
            for (int dx = 0; dx < width; dx++)
                occupied[anchorCol + dx, anchorRow + dy] = filled;
        }
    }
}

using System;
using System.Collections.Generic;
```

方块解压馆软件 V1.0.0 源代码

```
using UnityEngine;
namespace Model.UI
{
    [Serializable]
    public class BlockRelaxShapeDefinition
    {
        [Tooltip("唯一 ID; 选块池条目通过此 ID 引用形状。")]
        public int ShapeId;
        [Tooltip("编辑器/调试用名称, 可不填。")]
        public string DisplayName;
        [Tooltip("整块选块预制 (根物体名用于运行时脚印反查)。")]
        public GameObject Prefab;
        [Tooltip("仅作参考; 运行时可由脚印推算。")]
        public int BoardWidth = 1;
        [Tooltip("仅作参考; 运行时可由脚印推算。")]
        public int BoardHeight = 1;
        [Tooltip("相对锚点 (0,0) 的各 1×1 占格列表顺序须与预制根下格子子物体顺序一致。")]
        public List<Vector2Int> BoardCellOffsets = new();
    }

    [CreateAssetMenu(fileName = "BlockRelaxShapeTable", menuName = "App/BlockRelax/配置/选块形状表", order = 9)]
    public class BlockRelaxShapeTable : ScriptableObject
    {
        [SerializeField] private List<BlockRelaxShapeDefinition> _shapes = new();
        public IReadOnlyList<BlockRelaxShapeDefinition> Shapes => _shapes;
        public bool TryGet(int shapeId, out BlockRelaxShapeDefinition def)
        {
            def = null;
            if (_shapes == null)
                return false;
            for (int i = 0; i < _shapes.Count; i++)
            {
                var s = _shapes[i];
                if (s != null && s.ShapeId == shapeId)
                {
                    def = s;
                    return true;
                }
            }
            return false;
        }

        public bool TryGetByPrefabRootName(string pieceRootNameWithoutClone, out BlockRelaxShapeDefinition def)
        {
            def = null;
            if (string.IsNullOrEmpty(pieceRootNameWithoutClone) || _shapes == null)
                return false;
            for (int i = 0; i < _shapes.Count; i++)
            {
                var s = _shapes[i];
                if (s?.Prefab != null && s.Prefab.name == pieceRootNameWithoutClone)
                {
                    def = s;
                }
            }
        }
    }
}
```

```
        return true;
    }
}
return false;
}
public int GetCellCount(int shapeId)
{
    if (!TryGet(shapeId, out var def) || def.BoardCellOffsets == null)
        return 0;
    return def.BoardCellOffsets.Count;
}
public bool TryGetNormalizedFootprintCells(int shapeId, List<Vector2Int> into)
{
    into.Clear();
    if (!TryGet(shapeId, out var def) || def.BoardCellOffsets == null ||
def.BoardCellOffsets.Count == 0)
        return false;
    return TryCopyNormalizedFootprint(def.BoardCellOffsets, into);
}
public static bool TryCopyNormalizedFootprint(IReadOnlyList<Vector2Int> source,
List<Vector2Int> into)
{
    into.Clear();
    if (source == null || source.Count == 0)
        return false;
    int minX = int.MaxValue;
    int minY = int.MaxValue;
    for (int k = 0; k < source.Count; k++)
    {
        var c = source[k];
        if (c.x < minX)
            minX = c.x;
        if (c.y < minY)
            minY = c.y;
    }
    for (int k = 0; k < source.Count; k++)
    {
        var c = source[k];
        into.Add(new Vector2Int(c.x - minX, c.y - minY));
    }
    return into.Count > 0;
}
}
}
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Model.UI
{
    public enum BlockRelaxLevelCompletionMode
    {
        ItemObjectives = 0,
        TargetScore = 1,
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        ItemObjectivesAndScore = 2,
        ItemObjectivesOrScore = 3,
    }
    [Serializable]
    public class BlockRelaxLevelCondition
    {
        [Tooltip("收集目标类型（与 BlockRelaxCollectSpriteLibrary、棋盘子物体 Collect 一致；非果冻 BlockRelaxItemType）。")]
        public BlockRelaxCollectType CollectType = BlockRelaxCollectType.Collect01;
        [Tooltip("本关需要达成的该收集类型数量。")]
        public int RequiredCount = 1;
    }
    [Serializable]
    public class BlockRelaxLevelEntry
    {
        [Tooltip("关卡编号（展示或存档用）；与列表下标无关。")]
        public int LevelId = 1;
        [Tooltip("关卡名，可选。")]
        public string DisplayName = string.Empty;
        [Range(0f, 1f)]
        [Tooltip("选块池加权难度 0~1，对应 GameBlockRelaxModel.SelectPieceDifficulty01。")]
        public float SelectPieceDifficulty01 = 0.5f;
        [Tooltip("非空时本关改用该选块池；为空则用全局默认池。")]
        public BlockRelaxSelectPiecePool PoolOverride;
        [Tooltip("过关条件（须全部满足）；为空列表时不通过条件过关（仅死局失败）。")]
        public List<BlockRelaxLevelCondition> CompletionConditions = new();
        [Tooltip("过关方式：收集目标数量 / 目标分数 / 组合。注意：CompletionMode 仍为默认 ItemObjectives 且未填有效收集条（RequiredCount>0 且 CollectType≠None）时，若 TargetScore>0 则按「达到目标分」过关，以免只配了分数却不切枚导致无法结算。")]
        public BlockRelaxLevelCompletionMode CompletionMode = BlockRelaxLevelCompletionMode.ItemObjectives;
        [Tooltip("分数模式下需达到的分数；组合模式下与道具条件一同使用。")]
        public int TargetScore = 1000;
        [Tooltip("分数阶段权重，长度须与界面 GameBlockRelaxScoreTargetView 中 _textScoreTargets 数量一致；按权重将 TargetScore 拆成各段（界面显示每段结束时的累计分）。为空或长度不一致时按阶段数均分 TargetScore。")]
        public List<int> ScoreTargetStageWeights = new();
        [Tooltip("每次消除结算时，每个被清除格子计入的分数（≤0 时运行时使用默认 10）。")]
        public int ScorePerClearedCell = 10;
        [Tooltip("本关果冻贴图皮肤编号（如 01）；留空则用选块池上的 DefaultItemSpriteSkinId。对应资源名 BlockRelaxItemSprite_{皮肤}_01..05。")]
        public string ItemSpriteSkinId = string.Empty;
        [Tooltip("收集物图标皮肤编号（如 01），与 BlockRelaxCollectSpriteLibrary 中 SkinId 对应；留空则用本关 ItemSpriteSkinId；仍空则用库 DefaultSkinId。Collect 仅挂在果冻格子下。")]
        public string CollectSpriteSkinId = string.Empty;
        [Tooltip("无合法步时自动复活次数：选出两块最「密」的 3×3 区域，标记后清除其中已占格；0 表示关闭。")]
        [Min(0)]
        public int ReviveCountOnNoMoves = 3;
    }
    [CreateAssetMenu(fileName = "BlockRelaxLevelTable", menuName = "App/BlockRelax/配置/关卡表", order = 11)]
```

方块解压馆软件 V1.0.0 源代码

```
public class BlockRelaxLevelTable : ScriptableObject
{
    [SerializeField] private List<BlockRelaxLevelEntry> _levels = new();
    public IReadOnlyList<BlockRelaxLevelEntry> Levels => _levels;
    public int LevelCount => _levels != null ? _levels.Count : 0;
    public bool TryGetLevel(int zeroBasedIndex, out BlockRelaxLevelEntry entry)
    {
        entry = null;
        if (_levels == null || _levels.Count == 0)
            return false;
        int i = zeroBasedIndex;
        if (i < 0)
            i = 0;
        if (i >= _levels.Count)
            i = _levels.Count - 1;
        entry = _levels[i];
        return entry != null;
    }
}

using System;
using System.Collections.Generic;
using UnityEngine;
namespace Model.UI
{
    [Serializable]
    public class BlockRelaxCollectTypeWeightEntry
    {
        [Tooltip("收集物类型（与关卡 CollectType 一致）。")]
        public BlockRelaxCollectType CollectType = BlockRelaxCollectType.Collect01;
        [Tooltip("相对权重；≤0 时忽略。")]
        public float Weight = 1f;
    }

    [Serializable]
    public class BlockRelaxSelectPiecePoolEntry
    {
        [Tooltip("对应 BlockRelaxShapeTable 中的 ShapeId；可多行重复。")]
        public int ShapeId;
        [Tooltip("挂到 SelectItem 父节点下时的 localScale")]
        public Vector3 PreviewScale = new Vector3(0.35f, 0.35f, 0.35f);
        [Tooltip("相对权重（不必加总为 100）。同池内多条目加权抽样；≤0 时按最小有效权重处理。")]
        public float BaseWeight = 1f;
        [Tooltip("该形状落在棋盘上时，每个占格对应的道具类型（消除时计入关卡条件）。")]
        public BlockRelaxItemType ItemType = BlockRelaxItemType.JellyBlue;
        [Tooltip("每个脚印格独立掷骰：是否生成/显示 Collect 子物体，概率 0~1；≤0 表示关闭。")]
        [Range(0f, 1f)]
        public float CollectSpawnChancePerCell = 0f;
        [Tooltip("出现 Collect 时按权重抽样 CollectType；为空或无效时默认 Collect01。")]
        public List<BlockRelaxCollectTypeWeightEntry> CollectTypeWeights = new();
    }

    [CreateAssetMenu(fileName = "BlockRelaxSelectPiecePool", menuName = "App/BlockRelax/配置/")]
}
```


方块解压馆软件 V1.0.0 源代码

```
选块池”，order = 10)]
public class BlockRelaxSelectPiecePool : ScriptableObject
{
    [SerializeField] private List<BlockRelaxSelectPiecePoolEntry> _entries = new();
    [Tooltip(“形状定义表（脚印 + 预制）；必填。”)]
    [SerializeField] private BlockRelaxShapeTable _shapeTable;
    [Tooltip(“可选：按当前皮肤与道具类型替换选块/预览上 Image.sprite（命名见 BlockRelaxItemSpriteUtility）。”)]
    [SerializeField] private BlockRelaxItemSpriteLibrary _itemSpriteLibrary;
    [Tooltip(“关卡未填写 ItemSpriteSkinId 时使用的皮肤编号（如 01）。”)]
    [SerializeField] private string _defaultItemSpriteSkinId = "01";
    public BlockRelaxShapeTable ShapeTable => _shapeTable;
    public BlockRelaxItemSpriteLibrary ItemSpriteLibrary => _itemSpriteLibrary;
    public string DefaultItemSpriteSkinId => _defaultItemSpriteSkinId;
    [NonSerialized] private readonly List<int> _pickThreeValidIndicesScratch = new
List<int>(16);
    [NonSerialized] private readonly List<int> _pickThreeRemainingIndicesScratch = new
List<int>(16);
    [NonSerialized] private readonly List<float> _pickThreeWeightLineScratch = new
List<float>(16);
    [NonSerialized] private readonly List<Vector2Int> _normalizedShapeScratch = new
List<Vector2Int>(16);
    [NonSerialized] private readonly List<BlockRelaxSelectPiecePoolEntry>
_pickThreeTripletScratch = new List<BlockRelaxSelectPiecePoolEntry>(3);
    [NonSerialized] private readonly List<int> _placableEntryIndicesScratch = new
List<int>(16);
    private const int SolvabilityRetryAttempts = 6;
    public IReadOnlyList<BlockRelaxSelectPiecePoolEntry> Entries => _entries;
    public GameObject GetPrefabForEntry(BlockRelaxSelectPiecePoolEntry entry)
    {
        if (entry == null || _shapeTable == null)
            return null;
        return _shapeTable.TryGet(entry.ShapeId, out var def) ? def.Prefab : null;
    }
    public int GetShapeCellCount(BlockRelaxSelectPiecePoolEntry entry)
    {
        if (entry == null || _shapeTable == null)
            return 0;
        return _shapeTable.GetCellCount(entry.ShapeId);
    }
    public bool TryGetBoardFootprint(string pieceRootNameWithoutClone, out int width, out
int height)
    {
        width = height = 0;
        if (string.IsNullOrEmpty(pieceRootNameWithoutClone) || _shapeTable == null)
            return false;
        if (!_shapeTable.TryGetByPrefabRootName(pieceRootNameWithoutClone, out var def))
            return false;
        if (!_shapeTable.TryGetNormalizedFootprintCells(def.ShapeId,
_normalizedShapeScratch))
            return false;
        ComputeBBox(_normalizedShapeScratch, out width, out height);
        return width > 0 && height > 0;
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
    }
    public bool TryGetBoardFootprintCells(string pieceRootNameWithoutClone,
List<Vector2Int> into)
    {
        into.Clear();
        if (string.IsNullOrEmpty(pieceRootNameWithoutClone) || _shapeTable == null)
            return false;
        if (!_shapeTable.TryGetByPrefabRootName(pieceRootNameWithoutClone, out var def))
            return false;
        return _shapeTable.TryGetNormalizedFootprintCells(def.ShapeId, into);
    }
    public bool TryGetItemTypeForPrefabRoot(string pieceRootNameWithoutClone, out
BlockRelaxItemType itemType)
    {
        itemType = BlockRelaxItemType.JellyBlue;
        if (string.IsNullOrEmpty(pieceRootNameWithoutClone) || _entries == null)
            return false;
        for (int i = 0; i < _entries.Count; i++)
        {
            var e = _entries[i];
            var p = GetPrefabForEntry(e);
            if (p == null || p.name != pieceRootNameWithoutClone)
                continue;
            itemType = e.ItemType == BlockRelaxItemType.None ?
BlockRelaxItemType.JellyBlue : e.ItemType;
            return true;
        }
        return false;
    }
    public void PickThreeEntries(
        List<BlockRelaxSelectPiecePoolEntry> into,
        float selectPieceDifficulty01,
        bool[,] boardOccupied,
        int boardCols,
        int boardRows)
    {
        into.Clear();
        if (_entries == null || _entries.Count == 0)
            return;
        var validIndices = _pickThreeValidIndicesScratch;
        validIndices.Clear();
        for (int i = 0; i < _entries.Count; i++)
        {
            var e = _entries[i];
            if (e != null && IsEntryPlayable(e))
                validIndices.Add(i);
        }
        if (validIndices.Count == 0)
            return;
        int maxCells = GetMaxCellCountAmongEntryIndices(validIndices);
        bool useSolvabilityGuard =
            boardOccupied != null
            && boardCols > 0
```

方块解压馆软件 V1.0.0 源代码

```
        && boardRows > 0
        && boardOccupied.GetLength(0) >= boardCols
        && boardOccupied.GetLength(1) >= boardRows
        && BoardHasAnyEmptyCell(boardOccupied, boardCols, boardRows);
    if (!useSolvabilityGuard)
    {
        PickThreeWeightedTripletInto(into, selectPieceDifficulty01, validIndices,
maxCells);
        return;
    }
    var triplet = _pickThreeTripletScratch;
    for (int attempt = 0; attempt < SolvabilityRetryAttempts; attempt++)
    {
        triplet.Clear();
        PickThreeWeightedTripletInto(triplet, selectPieceDifficulty01, validIndices,
maxCells);
        if (TripletHasAtLeastOnePlaceableOnBoard(triplet, boardOccupied, boardCols,
boardRows))
        {
            for (int t = 0; t < triplet.Count; t++)
                into.Add(triplet[t]);
            return;
        }
    }
    BuildPlacableEntryIndices(boardOccupied, boardCols, boardRows, validIndices,
_placableEntryIndicesScratch);
    if (_placableEntryIndicesScratch.Count == 0)
    {
        PickThreeWeightedTripletInto(into, selectPieceDifficulty01, validIndices,
maxCells);
        return;
    }
    var weights = _pickThreeWeightLineScratch;
    int guaranteedEntryIndex = PickOneWeightedEntryIndexFromList(
        _placableEntryIndicesScratch,
        weights,
        selectPieceDifficulty01,
        maxCells);
    into.Add(_entries[guaranteedEntryIndex]);
    var remaining = _pickThreeRemainingIndicesScratch;
    for (int extra = 0; extra < 2; extra++)
    {
        remaining.Clear();
        for (int i = 0; i < validIndices.Count; i++)
            remaining.Add(validIndices[i]);
        int pickIdx = PickOneWeightedRemainingIndex(remaining, weights,
selectPieceDifficulty01, maxCells);
        into.Add(_entries[remaining[pickIdx]]);
        int last = remaining.Count - 1;
        if (pickIdx < last)
            remaining[pickIdx] = remaining[last];
        remaining.RemoveAt(last);
    }
}
```

```
}
private bool IsEntryPlayable(BlockRelaxSelectPiecePoolEntry entry)
{
    if (_shapeTable == null || entry == null)
        return false;
    if (!_shapeTable.TryGetValue(entry.ShapeId, out var def))
        return false;
    if (def.Prefab == null || def.BoardCellOffsets == null || def.BoardCellOffsets.Count
== 0)
        return false;
    return true;
}
private int GetMaxCellCountAmongEntryIndices(List<int> validEntryIndices)
{
    int maxC = 1;
    for (int i = 0; i < validEntryIndices.Count; i++)
    {
        int idx = validEntryIndices[i];
        if (idx < 0 || idx >= _entries.Count)
            continue;
        int c = GetShapeCellCount(_entries[idx]);
        if (c > maxC)
            maxC = c;
    }
    return maxC;
}
private bool TryCopyNormalizedOffsetsForEntry(BlockRelaxSelectPiecePoolEntry entry,
List<Vector2Int> into)
{
    if (entry == null || _shapeTable == null)
        return false;
    return _shapeTable.TryGetValue(entry.ShapeId, into);
}
private static void ComputeBBox(IReadOnlyList<Vector2Int> cells, out int bboxWidth, out
int bboxHeight)
{
    bboxWidth = bboxHeight = 0;
    if (cells == null || cells.Count == 0)
        return;
    int maxX = int.MinValue;
    int maxY = int.MinValue;
    for (int i = 0; i < cells.Count; i++)
    {
        var c = cells[i];
        if (c.x > maxX)
            maxX = c.x;
        if (c.y > maxY)
            maxY = c.y;
    }
    bboxWidth = maxX + 1;
    bboxHeight = maxY + 1;
}
private static bool BoardHasAnyEmptyCell(bool[, ] occupied, int boardCols, int boardRows)
```

```
{
    for (int c = 0; c < boardCols; c++)
    {
        for (int r = 0; r < boardRows; r++)
        {
            if (!occupied[c, r])
                return true;
        }
    }
    return false;
}

private void BuildPlacableEntryIndices(
    bool[,] occupied,
    int boardCols,
    int boardRows,
    List<int> validEntryIndices,
    List<int> placableOut)
{
    placableOut.Clear();
    for (int i = 0; i < validEntryIndices.Count; i++)
    {
        int idx = validEntryIndices[i];
        if (idx < 0 || idx >= _entries.Count)
            continue;
        var e = _entries[idx];
        if (!TryCopyNormalizedOffsetsForEntry(e, _normalizedShapeScratch))
            continue;
        if (BlockRelaxPieceShape.HasAnyPlacementForShape(occupied, boardCols,
boardRows, _normalizedShapeScratch))
            placableOut.Add(idx);
    }
}

private bool TripletHasAtLeastOnePlaceableOnBoard(
    List<BlockRelaxSelectPiecePoolEntry> triplet,
    bool[,] occupied,
    int boardCols,
    int boardRows)
{
    for (int i = 0; i < triplet.Count; i++)
    {
        var e = triplet[i];
        if (e == null)
            continue;
        if (!TryCopyNormalizedOffsetsForEntry(e, _normalizedShapeScratch))
            continue;
        if (BlockRelaxPieceShape.HasAnyPlacementForShape(occupied, boardCols,
boardRows, _normalizedShapeScratch))
            return true;
    }
    return false;
}

private void PickThreeWeightedTripletInto(
    List<BlockRelaxSelectPiecePoolEntry> dest,
```

方块解压馆软件 V1.0.0 源代码

```
float selectPieceDifficulty01,
List<int> validIndices,
int maxCells)
{
    dest.Clear();
    var remaining = _pickThreeRemainingIndicesScratch;
    remaining.Clear();
    for (int i = 0; i < validIndices.Count; i++)
        remaining.Add(validIndices[i]);
    var weights = _pickThreeWeightLineScratch;
    for (int slot = 0; slot < 3; slot++)
    {
        if (remaining.Count == 0)
        {
            for (int i = 0; i < validIndices.Count; i++)
                remaining.Add(validIndices[i]);
        }
        int pickRemainIndex = PickOneWeightedRemainingIndex(remaining, weights,
selectPieceDifficulty01, maxCells);
        int entryIndex = remaining[pickRemainIndex];
        dest.Add(_entries[entryIndex]);
        int last = remaining.Count - 1;
        if (pickRemainIndex < last)
            remaining[pickRemainIndex] = remaining[last];
        remaining.RemoveAt(last);
    }
}

private int PickOneWeightedEntryIndexFromList(
    List<int> entryIndicesIntoPool,
    List<float> weights,
    float difficulty01,
    int maxCellCountAmongValid)
{
    weights.Clear();
    float sum = 0f;
    for (int i = 0; i < entryIndicesIntoPool.Count; i++)
    {
        var e = _entries[entryIndicesIntoPool[i]];
        int cells = GetShapeCellCount(e);
        float w = BlockRelaxSelectPiecePoolWeights.ComputeEffectiveWeight(e,
difficulty01, maxCellCountAmongValid, cells);
        weights.Add(w);
        sum += w;
    }
    if (sum < 1e-6f)
        return entryIndicesIntoPool[UnityEngine.Random.Range(0,
entryIndicesIntoPool.Count)];
    float r = UnityEngine.Random.value * sum;
    float acc = 0f;
    for (int i = 0; i < weights.Count; i++)
    {
        acc += weights[i];
        if (r <= acc)
```

```
        return entryIndicesIntoPool[i];
    }
    return entryIndicesIntoPool[entryIndicesIntoPool.Count - 1];
}
private int PickOneWeightedRemainingIndex(
    List<int> remaining,
    List<float> weights,
    float difficulty01,
    int maxCellCountAmongValid)
{
    weights.Clear();
    float sum = 0f;
    for (int i = 0; i < remaining.Count; i++)
    {
        var e = _entries[remaining[i]];
        int cells = GetShapeCellCount(e);
        float w = BlockRelaxSelectPiecePoolWeights.ComputeEffectiveWeight(e,
difficulty01, maxCellCountAmongValid, cells);
        weights.Add(w);
        sum += w;
    }
    if (sum < 1e-6f)
        return UnityEngine.Random.Range(0, remaining.Count);
    float r = UnityEngine.Random.value * sum;
    float acc = 0f;
    for (int i = 0; i < weights.Count; i++)
    {
        acc += weights[i];
        if (r <= acc)
            return i;
    }
    return weights.Count - 1;
}
public void PickThreeEntriesWithoutBoard(List<BlockRelaxSelectPiecePoolEntry> into,
float selectPieceDifficulty01 = 0.5f) =>
    PickThreeEntries(into, selectPieceDifficulty01, null, 0, 0);
public static BlockRelaxCollectType
PickCollectTypeForSpawn(BlockRelaxSelectPiecePoolEntry entry)
{
    if (entry?.CollectTypeWeights == null || entry.CollectTypeWeights.Count == 0)
        return BlockRelaxCollectType.Collect01;
    float sum = 0f;
    for (int i = 0; i < entry.CollectTypeWeights.Count; i++)
    {
        var w = entry.CollectTypeWeights[i];
        if (w == null || w.CollectType == BlockRelaxCollectType.None || w.Weight <= 0f)
            continue;
        sum += w.Weight;
    }
    if (sum < 1e-6f)
        return BlockRelaxCollectType.Collect01;
    float r = UnityEngine.Random.value * sum;
    float acc = 0f;
```

方块解压馆软件 V1.0.0 源代码

```
        for (int i = 0; i < entry.CollectTypeWeights.Count; i++)
        {
            var w = entry.CollectTypeWeights[i];
            if (w == null || w.CollectType == BlockRelaxCollectType.None || w.Weight <= 0f)
                continue;
            acc += w.Weight;
            if (r <= acc)
                return w.CollectType;
        }
        return BlockRelaxCollectType.Collect01;
    }
}

using UnityEngine;
namespace Model.UI
{
    public static class BlockRelaxSelectPiecePoolWeights
    {
        public const float MinEffectiveWeight = 1e-4f;
        private const float MinShapeScaleForPow = 0.08f;
        private const float DifficultyExponentHalfRange = 1.15f;
        public static float ComputeEffectiveWeight(
            BlockRelaxSelectPiecePoolEntry entry,
            float difficulty01,
            int maxCellCountAmongValid,
            int shapeCellCount)
        {
            if (entry == null)
                return MinEffectiveWeight;
            float baseW = entry.BaseWeight < MinEffectiveWeight ? MinEffectiveWeight :
entry.BaseWeight;
            int cells = Mathf.Max(1, shapeCellCount);
            int denom = Mathf.Max(1, maxCellCountAmongValid - 1);
            float normalized = (cells - 1) / (float)denom;
            float shapeScale = Mathf.Max(MinShapeScaleForPow, normalized);
            float d = Mathf.Clamp01(difficulty01);
            float exponent = Mathf.Lerp(-DifficultyExponentHalfRange,
DifficultyExponentHalfRange, d);
            float w = baseW * Mathf.Pow(shapeScale, exponent);
            return w < MinEffectiveWeight ? MinEffectiveWeight : w;
        }
    }
}

using System;
using System.Collections.Generic;
using UnityEngine;
namespace Model.UI
{
    [Serializable]
    public class BlockRelaxItemSpriteSkinEntry
    {
        [Tooltip("皮肤编号, 如 01, 对应命名 BlockRelaxItemSprite_01_xx")]
        public string SkinId = "01";
    }
}
```


方块解压馆软件 V1.0.0 源代码

```
[Tooltip("长度 5: 下标 0~4 对应 BlockRelaxItemSprite_{SkinId}_01 .. _05")]
public Sprite[] Variants = new Sprite[BlockRelaxItemSpriteUtility.VariantCount];
}
[CreateAssetMenu(fileName = "BlockRelaxItemSpriteLibrary", menuName = "App/BlockRelax/配置/道具贴图皮肤库", order = 12)]
public class BlockRelaxItemSpriteLibrary : ScriptableObject
{
    [SerializeField] private List<BlockRelaxItemSpriteSkinEntry> _skins = new();
    public IReadOnlyList<BlockRelaxItemSpriteSkinEntry> Skins => _skins;
    public bool TryGetSprite(string skinId, BlockRelaxItemType itemType, out Sprite sprite)
    {
        sprite = null;
        if (_skins == null || _skins.Count == 0)
            return false;
        if (!BlockRelaxItemSpriteUtility.TryGetVariantIndexZeroBased(itemType, out int z))
            return false;
        string key = string.IsNullOrEmpty(skinId) ? "01" : skinId.Trim();
        for (int i = 0; i < _skins.Count; i++)
        {
            var row = _skins[i];
            if (row == null || string.IsNullOrEmpty(row.SkinId))
                continue;
            if (!string.Equals(row.SkinId.Trim(), key, StringComparison.Ordinal))
                continue;
            if (row.Variants == null || z >= row.Variants.Length)
                return false;
            sprite = row.Variants[z];
            return sprite != null;
        }
        return false;
    }
}
using System;
using System.Collections.Generic;
using System.Globalization;
using UnityEngine;
namespace Model.UI
{
    [Serializable]
    public class BlockRelaxCollectSpritePairEntry
    {
        [Tooltip("收集目标类型，与关卡 CompletionConditions 中的 CollectType 对应。")]
        public BlockRelaxCollectType CollectType = BlockRelaxCollectType.Collect01;
        [Tooltip("该类型在 HUD / 棋盘子物体上使用的图标。")]
        public Sprite Sprite;
    }
    [Serializable]
    public class BlockRelaxCollectSpriteSkinEntry
    {
        [Tooltip("皮肤编号，如 01；与关卡 CollectSpriteSkinId / ItemSpriteSkinId 对应。")]
        public string SkinId = "01";
        [Tooltip("各收集类型对应的图标；未列出的类型解析失败，调用方保持预制默认图。")]
```

方块解压馆软件 V1.0.0 源代码

```
        public List<BlockRelaxCollectSpritePairEntry> Pairs = new();
    }
    [CreateAssetMenu(fileName = "BlockRelaxCollectSpriteLibrary", menuName = "App/BlockRelax/
配置/收集物图标库", order = 13)]
    public class BlockRelaxCollectSpriteLibrary : ScriptableObject
    {
        [Tooltip("关卡未指定 CollectSpriteSkinId 且 ItemSpriteSkinId 为空时使用的皮肤编号。
")]
        [SerializeField] private string _defaultSkinId = "01";
        [SerializeField] private List<BlockRelaxCollectSpriteSkinEntry> _skins = new();
        public IReadOnlyList<BlockRelaxCollectSpriteSkinEntry> Skins => _skins;
        public string DefaultSkinId =>
            NormalizeSkinIdKey(_defaultSkinId) ?? "01";
        public static string NormalizeSkinIdKey(string skinId)
        {
            if (string.IsNullOrEmpty(skinId))
                return null;
            string t = skinId.Trim();
            if (int.TryParse(t, NumberStyles.Integer, CultureInfo.InvariantCulture, out int n)
&& n >= 0)
                return n.ToString("D2", CultureInfo.InvariantCulture);
            return t;
        }
        public bool TryGetSprite(string skinId, BlockRelaxCollectType collectType, out Sprite
sprite)
        {
            sprite = null;
            if (collectType == BlockRelaxCollectType.None)
                return false;
            if (_skins == null || _skins.Count == 0)
                return false;
            string primary = string.IsNullOrEmpty(skinId)
                ? DefaultSkinId
                : NormalizeSkinIdKey(skinId) ?? skinId.Trim();
            if (TryGetSpriteForExactSkinKey(primary, collectType, out sprite))
                return true;
            string trimmed = string.IsNullOrEmpty(skinId) ? null : skinId.Trim();
            if (!string.IsNullOrEmpty(trimmed)
&& !string.Equals(trimmed, primary, StringComparison.Ordinal)
&& TryGetSpriteForExactSkinKey(trimmed, collectType, out sprite))
                return true;
            return false;
        }
        private bool TryGetSpriteForExactSkinKey(string key, BlockRelaxCollectType collectType,
out Sprite sprite)
        {
            sprite = null;
            if (string.IsNullOrEmpty(key))
                return false;
            for (int s = 0; s < _skins.Count; s++)
            {
                var row = _skins[s];
                if (row == null || string.IsNullOrEmpty(row.SkinId))
```

```
        continue;
    if (!string.Equals(row.SkinId.Trim(), key, StringComparison.Ordinal))
        continue;
    var pairs = row.Pairs;
    if (pairs == null)
        return false;
    for (int i = 0; i < pairs.Count; i++)
    {
        var p = pairs[i];
        if (p == null || p.CollectType != collectType)
            continue;
        sprite = p.Sprite;
        return sprite != null;
    }
    return false;
}
return false;
}
}
}
using Storage;
using UnityEngine;
namespace Model.UI
{
    public static class BlockRelaxProgress
    {
        private const string StorageKey = "game.blockrelax.runtimeLevelIndex";
        private const string LegacyBlockBlastStorageKey = "game.blockblast.runtimeLevelIndex";
        private const string LegacyPlayerPrefsKey = "BlockRelax.RuntimeLevelIndex";
        private const string LegacyBlockBlastPlayerPrefsKey = "BlockBlast.RuntimeLevelIndex";
        public static int GetCurrentLevelIndex()
        {
            MigrateLegacyPlayerPrefsIfNeeded();
            return Mathf.Max(0, StorageManager.GetInt(StorageKey, 0));
        }
        public static void SetCurrentLevelIndex(int zeroBasedIndex)
        {
            MigrateLegacyPlayerPrefsIfNeeded();
            StorageManager.SetInt(StorageKey, Mathf.Max(0, zeroBasedIndex));
        }
        public static void AdvanceAfterLevelWin(int levelTableCount)
        {
            if (levelTableCount <= 0)
                return;
            int next = GetCurrentLevelIndex() + 1;
            if (next >= levelTableCount)
                next = levelTableCount - 1;
            SetCurrentLevelIndex(next);
        }
        public static void ClearSavedProgress()
        {
            StorageManager.DeleteIfExists(StorageKey);
            StorageManager.DeleteIfExists(LegacyBlockBlastStorageKey);
        }
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        if (PlayerPrefs.HasKey(LegacyPlayerPrefsKey))
        {
            PlayerPrefs.DeleteKey(LegacyPlayerPrefsKey);
            PlayerPrefs.Save();
        }
        if (PlayerPrefs.HasKey(LegacyBlockBlastPlayerPrefsKey))
        {
            PlayerPrefs.DeleteKey(LegacyBlockBlastPlayerPrefsKey);
            PlayerPrefs.Save();
        }
    }
    private static void MigrateLegacyPlayerPrefsIfNeeded()
    {
        if (StorageManager.HasKey(StorageKey))
            return;
        if (StorageManager.HasKey(LegacyBlockBlastStorageKey))
        {
            int v = Mathf.Max(0, StorageManager.GetInt(LegacyBlockBlastStorageKey, 0));
            StorageManager.SetInt(StorageKey, v);
            StorageManager.DeleteIfExists(LegacyBlockBlastStorageKey);
            return;
        }
        if (PlayerPrefs.HasKey(LegacyBlockBlastPlayerPrefsKey))
        {
            int v = Mathf.Max(0, PlayerPrefs.GetInt(LegacyBlockBlastPlayerPrefsKey, 0));
            StorageManager.SetInt(StorageKey, v);
            PlayerPrefs.DeleteKey(LegacyBlockBlastPlayerPrefsKey);
            PlayerPrefs.Save();
            return;
        }
        if (!PlayerPrefs.HasKey(LegacyPlayerPrefsKey))
            return;
        int v2 = Mathf.Max(0, PlayerPrefs.GetInt(LegacyPlayerPrefsKey, 0));
        StorageManager.SetInt(StorageKey, v2);
        PlayerPrefs.DeleteKey(LegacyPlayerPrefsKey);
        PlayerPrefs.Save();
    }
}

using FW.Base;
using FW.Unity;
using UnityEngine;
namespace Model.UI
{
    public partial class GameBlockRelaxModel
    {
        public const int BoardSize = 8;
        public BlockRelaxSelectPiecePool SelectPiecePool { get; private set; }
        public float SelectPieceDifficulty01 { get; set; } = 0.5f;
        public int BlockRelaxLevelIndex { get; private set; }
        public bool[,] BoardOccupied { get; private set; }
        public int[,] BoardItemTypes { get; private set; }
        public int CurrentScore { get; private set; }
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
private int _selectTrayPiecesConsumedThisCycle;
private int _consecutivePlacementEliminationCount;
public          int          ConsecutivePlacementEliminationCount          =>
_consecutivePlacementEliminationCount;
private string _itemSpriteSkinId = "01";
public string ItemSpriteSkinId => _itemSpriteSkinId;
public void AssignSelectPiecePool(BlockRelaxSelectPiecePool pool) => SelectPiecePool =
pool;
public void SyncItemSpriteSkin(BlockRelaxLevelEntry levelEntry)
{
    string poolDefault =
        SelectPiecePool                                != null
        && !string.IsNullOrEmpty(SelectPiecePool.DefaultItemSpriteSkinId)
            ? SelectPiecePool.DefaultItemSpriteSkinId.Trim()
            : "01";
    if (levelEntry != null && !string.IsNullOrEmpty(levelEntry.ItemSpriteSkinId))
        _itemSpriteSkinId = levelEntry.ItemSpriteSkinId.Trim();
    else
        _itemSpriteSkinId = poolDefault;
}
public bool TryGetItemSprite(BlockRelaxItemType itemType, out Sprite sprite)
{
    sprite = null;
    var lib = SelectPiecePool?.ItemSpriteLibrary;
    if (lib == null)
        return false;
    return lib.TryGetSprite(_itemSpriteSkinId, itemType, out sprite);
}
public void ApplyLevelFromEntry(BlockRelaxLevelEntry entry)
{
    ResetObjectiveProgressForNewLevel(entry);
    if (entry == null)
        return;
    SelectPieceDifficulty01 = Mathf.Clamp01(entry.SelectPieceDifficulty01);
    if (entry.PoolOverride != null)
        AssignSelectPiecePool(entry.PoolOverride);
}
public void ResetSelectTrayConsumeCycle() => _selectTrayPiecesConsumedThisCycle = 0;
public bool TryConsumeSelectTrayPieceAndShouldRerollAllThreeSlots()
{
    _selectTrayPiecesConsumedThisCycle++;
    if (_selectTrayPiecesConsumedThisCycle < 3)
        return false;
    _selectTrayPiecesConsumedThisCycle = 0;
    return true;
}
public void OnPlacementWithLineElimination()
{
    _consecutivePlacementEliminationCount          =
    Mathf.Min(_consecutivePlacementEliminationCount + 1, 999);
}
public void OnPlacementWithoutLineElimination()
{

```

方块解压馆软件 V1.0.0 源代码

```
        _consecutivePlacementEliminationCount = 0;
    }
    private void ResetConsecutiveEliminationCombo()
    {
        _consecutivePlacementEliminationCount = 0;
    }
    public static GameBlockRelaxModel Create(BlockRelaxSelectPiecePool selectPiecePool =
null, int blockBlastLevelIndex = -1)
    {
        var m = new GameBlockRelaxModel();
        m.SelectPiecePool = selectPiecePool;
        int idx = blockBlastLevelIndex >= 0 ? blockBlastLevelIndex :
BlockRelaxProgress.GetCurrentLevelIndex();
        m.BlockRelaxLevelIndex = Mathf.Max(0, idx);
        m.BoardOccupied = new bool[BoardSize, BoardSize];
        m.BoardItemTypes = new int[BoardSize, BoardSize];
        return m;
    }
}
}
using System.Collections.Generic;
using UnityEngine;
namespace Model.UI
{
    public partial class GameBlockRelaxModel
    {
        public bool TryPlaceShapeAt(
            int anchorCol,
            int anchorRow,
            IReadOnlyList<Vector2Int> shapeCellsRelative,
            int boardCols,
            int boardRows,
            BlockRelaxItemType itemType)
        {
            if (BoardOccupied == null || BoardItemTypes == null || shapeCellsRelative == null
|| shapeCellsRelative.Count == 0)
                return false;
            if (!BlockRelaxPieceShape.CanPlace(BoardOccupied, boardCols, boardRows, anchorCol,
anchorRow, shapeCellsRelative))
                return false;
            BlockRelaxPieceShape.ApplyPlace(BoardOccupied, anchorCol, anchorRow,
shapeCellsRelative);
            int tid = itemType == BlockRelaxItemType.None ? (int)BlockRelaxItemType.JellyBlue :
(int)itemType;
            for (int i = 0; i < shapeCellsRelative.Count; i++)
            {
                var o = shapeCellsRelative[i];
                int c = anchorCol + o.x;
                int r = anchorRow + o.y;
                BoardItemTypes[c, r] = tid;
            }
            return true;
        }
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
public void RevertShapePlace(
    int anchorCol,
    int anchorRow,
    IReadOnlyList<Vector2Int> shapeCellsRelative)
{
    if (BoardOccupied == null || shapeCellsRelative == null || shapeCellsRelative.Count
== 0)
        return;
    for (int i = 0; i < shapeCellsRelative.Count; i++)
    {
        var o = shapeCellsRelative[i];
        int c = anchorCol + o.x;
        int r = anchorRow + o.y;
        if (c < 0 || r < 0 || c >= BoardOccupied.GetLength(0) || r >=
BoardOccupied.GetLength(1))
            continue;
        BoardOccupied[c, r] = false;
        if (BoardItemTypes != null)
            BoardItemTypes[c, r] = 0;
    }
}

}

}

using System.Collections.Generic;
using UnityEngine;
namespace Model.UI
{
    public partial class GameBlockRelaxModel
    {
        private bool[,] _eliminateMarkScratch;
        private bool[,] _boardSimScratch;
        private void EnsureEliminateMarkScratch(int cols, int rows)
        {
            if (_eliminateMarkScratch != null
                && _eliminateMarkScratch.GetLength(0) == cols
                && _eliminateMarkScratch.GetLength(1) == rows)
                return;
            _eliminateMarkScratch = new bool[cols, rows];
        }
        private void EnsureBoardSimScratch(int cols, int rows)
        {
            if (_boardSimScratch != null
                && _boardSimScratch.GetLength(0) == cols
                && _boardSimScratch.GetLength(1) == rows)
                return;
            _boardSimScratch = new bool[cols, rows];
        }
        private static void FillEliminationLineMarks(bool[,] occupied, int boardCols, int
boardRows, bool[,] marksOut)
        {
            for (int r = 0; r < boardRows; r++)
            {
                for (int c = 0; c < boardCols; c++)
```

```
        marksOut[c, r] = false;
    }
    for (int r = 0; r < boardRows; r++)
    {
        bool rowFull = true;
        for (int c = 0; c < boardCols; c++)
        {
            if (!occupied[c, r])
            {
                rowFull = false;
                break;
            }
        }
        if (!rowFull)
            continue;
        for (int c = 0; c < boardCols; c++)
            marksOut[c, r] = true;
    }
    for (int c = 0; c < boardCols; c++)
    {
        bool colFull = true;
        for (int r = 0; r < boardRows; r++)
        {
            if (!occupied[c, r])
            {
                colFull = false;
                break;
            }
        }
        if (!colFull)
            continue;
        for (int r = 0; r < boardRows; r++)
            marksOut[c, r] = true;
    }
}

private static void WriteMarkedCellsToList(bool[,] marks, int boardCols, int boardRows,
List<Vector2Int> listOut)
{
    listOut.Clear();
    for (int c = 0; c < boardCols; c++)
    {
        for (int r = 0; r < boardRows; r++)
        {
            if (marks[c, r])
                listOut.Add(new Vector2Int(c, r));
        }
    }
}

public bool TryCollectEliminationCellsFromBoard(
    bool[,] occupied,
    int boardCols,
    int boardRows,
    List<Vector2Int> eliminationCellsOut)
```


方块解压馆软件 V1.0.0 源代码

```
{
    eliminationCellsOut.Clear();
    if (occupied == null || boardCols <= 0 || boardRows <= 0)
        return false;
    if (occupied.GetLength(0) < boardCols || occupied.GetLength(1) < boardRows)
        return false;
    EnsureEliminateMarkScratch(boardCols, boardRows);
    FillEliminationLineMarks(occupied, boardCols, boardRows, _eliminateMarkScratch);
    WriteMarkedCellsToList(_eliminateMarkScratch, boardCols, boardRows,
eliminationCellsOut);
    return eliminationCellsOut.Count > 0;
}

public int CountFullLinesOnCurrentBoard(int boardCols, int boardRows)
{
    if (BoardOccupied == null || boardCols <= 0 || boardRows <= 0)
        return 0;
    if (BoardOccupied.GetLength(0) < boardCols || BoardOccupied.GetLength(1) <
boardRows)
        return 0;
    int fullRows = 0;
    for (int r = 0; r < boardRows; r++)
    {
        bool rowFull = true;
        for (int c = 0; c < boardCols; c++)
        {
            if (!BoardOccupied[c, r])
            {
                rowFull = false;
                break;
            }
        }
        if (rowFull)
            fullRows++;
    }
    int fullCols = 0;
    for (int c = 0; c < boardCols; c++)
    {
        bool colFull = true;
        for (int r = 0; r < boardRows; r++)
        {
            if (!BoardOccupied[c, r])
            {
                colFull = false;
                break;
            }
        }
        if (colFull)
            fullCols++;
    }
    return fullRows + fullCols;
}

public bool TryGetEliminationCellsAfterHypotheticalPlace(
    bool[,] baseOccupied,
```

方块解压馆软件 V1.0.0 源代码

```
        int boardCols,
        int boardRows,
        IReadOnlyList<Vector2Int> placementAbsoluteCells,
        List<Vector2Int> eliminationCellsOut)
    {
        eliminationCellsOut.Clear();
        if (baseOccupied == null || placementAbsoluteCells == null ||
placementAbsoluteCells.Count == 0)
            return false;
        if (baseOccupied.GetLength(0) < boardCols || baseOccupied.GetLength(1) < boardRows)
            return false;
        EnsureBoardSimScratch(boardCols, boardRows);
        for (int c = 0; c < boardCols; c++)
        {
            for (int r = 0; r < boardRows; r++)
                _boardSimScratch[c, r] = baseOccupied[c, r];
        }
        for (int i = 0; i < placementAbsoluteCells.Count; i++)
        {
            var p = placementAbsoluteCells[i];
            if (p.x < 0 || p.y < 0 || p.x >= boardCols || p.y >= boardRows)
                return false;
            _boardSimScratch[p.x, p.y] = true;
        }
        return TryCollectEliminationCellsFromBoard(_boardSimScratch, boardCols, boardRows,
eliminationCellsOut);
    }

    public bool TryEliminateFullRowsAndColumns(
        int boardCols,
        int boardRows,
        List<Vector2Int> clearedBoardCellsOut,
        int fullLineCountForScore,
        int consecutiveComboIncludingThis)
    {
        clearedBoardCellsOut.Clear();
        if (BoardOccupied == null || boardCols <= 0 || boardRows <= 0)
            return false;
        if (BoardOccupied.GetLength(0) < boardCols || BoardOccupied.GetLength(1) <
boardRows)
            return false;
        if (!TryCollectEliminationCellsFromBoard(BoardOccupied, boardCols, boardRows,
clearedBoardCellsOut))
            return false;
        AddEliminationScore(
            clearedBoardCellsOut.Count,
            fullLineCountForScore,
            consecutiveComboIncludingThis);
        for (int i = 0; i < clearedBoardCellsOut.Count; i++)
        {
            var p = clearedBoardCellsOut[i];
            BoardOccupied[p.x, p.y] = false;
            if (BoardItemTypes != null)
                BoardItemTypes[p.x, p.y] = 0;
        }
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        }
        return true;
    }
}

using System;
using System.Collections.Generic;
using FW.Base;
using UnityEngine;
namespace Model.UI
{
    public partial class GameBlockRelaxModel
    {
        private BlockRelaxLevelEntry _activeLevelEntry;
        private readonly Dictionary<BlockRelaxCollectType, int> _collectObjectiveProgress =
new();
        [NonSerialized] private readonly HashSet<BlockRelaxCollectType>
_activeCollectObjectiveTypesScratch = new();
        [NonSerialized] private readonly List<BlockRelaxCollectType>
_collectSpawnPickListScratch = new List<BlockRelaxCollectType>(8);
        public void GetActiveCollectObjectiveTypesNonAlloc(HashSet<BlockRelaxCollectType>
into)
        {
            into.Clear();
            if (_activeLevelEntry?.CompletionConditions == null)
                return;
            for (int i = 0; i < _activeLevelEntry.CompletionConditions.Count; i++)
            {
                var c = _activeLevelEntry.CompletionConditions[i];
                if (c == null || c.RequiredCount <= 0 || c.CollectType ==
BlockRelaxCollectType.None)
                    continue;
                into.Add(c.CollectType);
            }
        }
        public bool TryPickCollectTypeForSpawnForActiveLevel(BlockRelaxSelectPiecePoolEntry
entry, out BlockRelaxCollectType collectType)
        {
            collectType = BlockRelaxCollectType.None;
            if (entry == null)
                return false;
            if (_activeLevelEntry != null
                && _activeLevelEntry.CompletionMode ==
BlockRelaxLevelCompletionMode.TargetScore)
                return false;
            var allowed = _activeCollectObjectiveTypesScratch;
            GetActiveCollectObjectiveTypesNonAlloc(allowed);
            if (allowed.Count == 0)
                return false;
            var weights = entry.CollectTypeWeights;
            float sum = 0f;
            if (weights != null && weights.Count > 0)
            {

```

方块解压馆软件 V1.0.0 源代码

```
        for (int i = 0; i < weights.Count; i++)
        {
            var w = weights[i];
            if (w == null || w.Weight <= 0f || w.CollectType ==
BlockRelaxCollectType.None)
                continue;
            if (!allowed.Contains(w.CollectType))
                continue;
            sum += w.Weight;
        }
        if (sum >= 1e-6f)
        {
            float r = UnityEngine.Random.value * sum;
            float acc = 0f;
            for (int i = 0; i < weights.Count; i++)
            {
                var w = weights[i];
                if (w == null || w.Weight <= 0f || w.CollectType ==
BlockRelaxCollectType.None)
                    continue;
                if (!allowed.Contains(w.CollectType))
                    continue;
                acc += w.Weight;
                if (r <= acc)
                {
                    collectType = w.CollectType;
                    return true;
                }
            }
            return false;
        }
    }
    var list = _collectSpawnPickListScratch;
    list.Clear();
    foreach (var t in allowed)
        list.Add(t);
    if (list.Count == 0)
        return false;
    collectType = list[UnityEngine.Random.Range(0, list.Count)];
    return true;
}

public BlockRelaxLevelEntry ActiveLevelEntry => _activeLevelEntry;
public int GetCollectObjectiveProgress(BlockRelaxCollectType collectType)
{
    if (collectType == BlockRelaxCollectType.None)
        return 0;
    return _collectObjectiveProgress.TryGetValue(collectType, out var n) ? n : 0;
}

private void ResetObjectiveProgressForNewLevel(BlockRelaxLevelEntry entry)
{
    _activeLevelEntry = entry;
    _collectObjectiveProgress.Clear();
    CurrentScore = 0;
}
```

方块解压馆软件 V1.0.0 源代码

```
ResetConsecutiveEliminationCombo();
ResetRevivesForNewLevel(entry);
}
private void AddEliminationScore(
    int clearedCellCount,
    int fullLineCountForScore,
    int consecutiveComboIncludingThis)
{
    if (clearedCellCount <= 0)
        return;
    int per = _activeLevelEntry != null && _activeLevelEntry.ScorePerClearedCell > 0
        ? _activeLevelEntry.ScorePerClearedCell
        : 10;
    float baseScore = clearedCellCount * per;
    var cfg = ConfigManager.Get<GameConfigData>(ConfigAddresses.GameConfig);
    float tier = cfg != null
        ? cfg.ResolveBlockRelaxLineClearScoreMultiplier(fullLineCountForScore)
        : DefaultBlockRelaxLineScoreMultiplierFallback(fullLineCountForScore);
    float combo = cfg != null
        ?
        cfg.ResolveBlockRelaxElimComboScoreMultiplier(consecutiveComboIncludingThis)
        : (consecutiveComboIncludingThis >= 2 ? 1.5f : 1f);
    CurrentScore += Mathf.RoundToInt(baseScore * tier * combo);
}
private static float DefaultBlockRelaxLineScoreMultiplierFallback(int fullLineCount)
{
    if (fullLineCount < 1)
        return 1f;
    if (fullLineCount >= 3)
        return 2f;
    if (fullLineCount >= 2)
        return 1.5f;
    return 1f;
}
public void RegisterCollectObjectiveProgress(BlockRelaxCollectType collectType, int
delta = 1)
{
    if (collectType == BlockRelaxCollectType.None || delta <= 0)
        return;
    _collectObjectiveProgress.TryGetValue(collectType, out var n);
    _collectObjectiveProgress[collectType] = n + delta;
}
public bool AreLevelCompletionConditionsSatisfied()
{
    if (_activeLevelEntry?.CompletionConditions == null ||
_activeLevelEntry.CompletionConditions.Count == 0)
        return false;
    var need = new Dictionary<BlockRelaxCollectType, int>();
    foreach (var cond in _activeLevelEntry.CompletionConditions)
    {
        if (cond.RequiredCount <= 0 || cond.CollectType == BlockRelaxCollectType.None)
            continue;
        need.TryGetValue(cond.CollectType, out var sum);
```

方块解压馆软件 V1.0.0 源代码

```
        need[cond.CollectType] = sum + cond.RequiredCount;
    }
    if (need.Count == 0)
        return false;
    foreach (var kv in need)
    {
        _collectObjectiveProgress.TryGetValue(kv.Key, out var got);
        if (got < kv.Value)
            return false;
    }
    return true;
}

private bool IsTargetScoreReached()
{
    if (_activeLevelEntry == null)
        return false;
    int need = _activeLevelEntry.TargetScore;
    if (need <= 0)
        return false;
    return CurrentScore >= need;
}

private bool HasActiveItemObjectiveRequirements()
{
    if (_activeLevelEntry?.CompletionConditions == null)
        return false;
    var list = _activeLevelEntry.CompletionConditions;
    for (int i = 0; i < list.Count; i++)
    {
        var c = list[i];
        if (c != null && c.RequiredCount > 0 && c.CollectType !=
BlockRelaxCollectType.None)
            return true;
    }
    return false;
}

private void AccumulateCollectProgressTotals(out int sumGot, out int sumNeed)
{
    sumGot = 0;
    sumNeed = 0;
    if (_activeLevelEntry?.CompletionConditions == null)
        return;
    var need = new Dictionary<BlockRelaxCollectType, int>();
    foreach (var cond in _activeLevelEntry.CompletionConditions)
    {
        if (cond == null || cond.RequiredCount <= 0 || cond.CollectType ==
BlockRelaxCollectType.None)
            continue;
        need.TryGetValue(cond.CollectType, out var n);
        need[cond.CollectType] = n + cond.RequiredCount;
    }
    foreach (var kv in need)
    {
        sumNeed += kv.Value;
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        _collectObjectiveProgress.TryGetValue(kv.Key, out var got);
        sumGot += Mathf.Min(got, kv.Value);
    }
}

public void GetActiveLevelGoalProgressForUi(out float normalized01, out string
progressLabel)
{
    normalized01 = 0f;
    progressLabel = string.Empty;
    if (_activeLevelEntry == null)
        return;
    var entry = _activeLevelEntry;
    int targetScore = Mathf.Max(0, entry.TargetScore);
    if (_blockRelaxItemCollectPrefab == null)
    {
#if UNITY_EDITOR || DEVELOPMENT_BUILD
        if (!warnedMissingCollectPrefab)
        {
            warnedMissingCollectPrefab = true;
            UnityEngine.Debug.LogWarning(
                "[GameBlockRelaxView] 选块需显示 Collect，但
BlockRelaxItemCollect 预制尚未加载（请在 RefreshSelectPieces 前 await
EnsureBlockRelaxItemCollectPrefabLoadedAsync），已跳过本块掷骰结果。");
        }
#endif
        continue;
    }
    var cellRt = cellRoots[i];
    if (!model.TryPickCollectTypeForSpawnForActiveLevel(entry, out var pick) ||
pick == BlockRelaxCollectType.None)
        continue;
    ReleaseRuntimeCollectsUnderPieceRoot(cellRt);
    var collect = RentBlockRelaxItemCollectView();
    if (collect == null)
        continue;
    AttachCollectUnderParentKeepPrefabLayout(collect, cellRt);
    collect.ConfigureCollect(model, pick);
    if (collect.TryGetComponent<Graphic>(out var g))
        g.raycastTarget = false;
    }
}

private void RestoreTrayCellsPooledCollectsAfterPlaceRollback(
    IReadOnlyList<RectTransform> cellRoots,
    IReadOnlyList<BlockRelaxCollectType> types,
    GameBlockRelaxModel model)
{
    if (cellRoots == null || types == null || model == null)
        return;
    int n = Mathf.Min(cellRoots.Count, types.Count);
    for (int i = 0; i < n; i++)
    {
        var pick = types[i];
        if (pick == BlockRelaxCollectType.None)
```

```
        continue;
    var cellRt = cellRoots[i];
    if (cellRt == null)
        continue;
    ReleaseRuntimeCollectsUnderPieceRoot(cellRt);
    var collect = RentBlockRelaxItemCollectView();
    if (collect == null)
        continue;
    AttachCollectUnderParentKeepPrefabLayout(collect, cellRt);
    collect.ConfigureCollect(model, pick);
    if (collect.TryGetComponent<Graphic>(out var g))
        g.raycastTarget = false;
    }
}

private BlockRelaxCollectType StealRuntimeCollectTypeFromCellAndReleaseToPool(RectTransform cellRt)
{
    if (cellRt == null)
        return BlockRelaxCollectType.None;
    var v = cellRt.GetComponentInChildren<BlockRelaxItemCollectView>(true);
    if (v == null || !v.IsViewPoolManaged)
        return BlockRelaxCollectType.None;
    var t = v.RuntimeCollectType;
    ReleaseBlockRelaxItemCollectViewToPool(v);
    return t == BlockRelaxCollectType.None ? BlockRelaxCollectType.None : t;
}

private void AttachPooledCollectToPlacedCellClone(
    RectTransform cloneRt,
    BlockRelaxCollectType type,
    GameBlockRelaxModel model)
{
    if (cloneRt == null || model == null || type == BlockRelaxCollectType.None)
        return;
    ReleaseRuntimeCollectsUnderPieceRoot(cloneRt);
    var collect = RentBlockRelaxItemCollectView();
    if (collect == null)
        return;
    AttachCollectUnderParentKeepPrefabLayout(collect, cloneRt);
    collect.ConfigureCollect(model, type);
    if (collect.TryGetComponent<Graphic>(out var g))
        g.raycastTarget = false;
}

private void ReleaseRuntimeCollectsUnderPieceRoot(RectTransform pieceRoot)
{
    if (pieceRoot == null)
        return;
    var scratch = _blockRelaxItemCollectViewsScratch;
    scratch.Clear();
    pieceRoot.GetComponentsInChildren(true, scratch);
    for (int i = 0; i < scratch.Count; i++)
    {
        var v = scratch[i];
        if (v != null && v.IsViewPoolManaged)
```


方块解压馆软件 V1.0.0 源代码

```
        ReleaseBlockRelaxItemCollectViewToPool(v);
    }
    scratch.Clear();
}
private static Transform TryGetPlacedBlockRelaxItemPrefabCloneRootUnderSlot(Image
slot)
{
    if (slot == null)
        return null;
    var rt = slot.rectTransform;
    if (rt.childCount == 0)
        return null;
    if (rt.childCount == 1)
        return rt.GetChild(0);
    const string placedPrefix = "PlacedPieceCell_";
    for (int i = 0; i < rt.childCount; i++)
    {
        var c = rt.GetChild(i);
        if (c.name.StartsWith(placedPrefix, StringComparison.Ordinal))
            return c;
    }
    return rt.GetChild(rt.childCount - 1);
}
private static BlockRelaxItemCollectView
TryFindPoolManagedCollectOnPlacedBlockRelaxItemClone(Image slot)
{
    var cloneRoot = TryGetPlacedBlockRelaxItemPrefabCloneRootUnderSlot(slot);
    if (cloneRoot == null)
        return null;
    var c = cloneRoot.GetComponentInChildren<BlockRelaxItemCollectView>(true);
    return IsPlacedCollectDetachable(c) ? c : null;
}
private static bool IsPlacedCollectDetachable(BlockRelaxItemCollectView v) =>
    v != null && v.RuntimeCollectType != BlockRelaxCollectType.None &&
v.IsViewPoolManaged;
public void DetachPlacedCollectsFromOccupiedForFly(IReadOnlyList<Vector2Int>
occupiedBoardCells)
{
    _detachedPlacedCollectFlyScratch.Clear();
    if (_gridSlots == null || occupiedBoardCells == null || occupiedBoardCells.Count ==
0)
    {
        #if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log(
                $"[BlockRelaxCollect] Detach abort: gridSlotsNull={_gridSlots == null}
listNull={occupiedBoardCells == null} count={occupiedBoardCells?.Count ?? 0}");
        #endif
        return;
    }
    var flyParent = GetCollectFlyLayerParent();
    if (flyParent == null)
    {
        #if UNITY_EDITOR || DEVELOPMENT_BUILD
```

方块解压馆软件 V1.0.0 源代码

```
UnityEngine.Debug.Log("[BlockRelaxCollect] Detach abort:
GetCollectFlyLayerParent() == null");
#endif
    return;
}
#if UNITY_EDITOR || DEVELOPMENT_BUILD
    UnityEngine.Debug.Log(
        "[BlockRelaxCollect] Detach begin cells={occupiedBoardCells.Count}
        flyParent={flyParent.name}");
#endif
    for (int i = 0; i < occupiedBoardCells.Count; i++)
    {
        var brd = occupiedBoardCells[i];
        var slot = GetSlot(brd.x, brd.y);
        if (slot == null)
        {
            #if UNITY_EDITOR || DEVELOPMENT_BUILD
                UnityEngine.Debug.Log($"[BlockRelaxCollect] Detach skip
                cell=({brd.x}, {brd.y}): GetSlot null");
            #endif
            continue;
        }
        var collect = TryFindPoolManagedCollectOnPlacedBlockRelaxItemClone(slot);
        #if UNITY_EDITOR || DEVELOPMENT_BUILD
            if (collect == null)
            {
                var srt = slot.rectTransform;
                var cr = TryGetPlacedBlockRelaxItemPrefabCloneRootUnderSlot(slot);
                var raw = cr != null ?
                cr.GetComponentInChildren<BlockRelaxItemCollectView>(true) : null;
                var sb = new StringBuilder();
                for (int k = 0; k < srt.childCount && k < 16; k++)
                    sb.Append(' ').Append(srt.GetChild(k).name).Append(' ');
                UnityEngine.Debug.Log(
                    "[BlockRelaxCollect] Detach skip cell=({brd.x}, {brd.y})
                    slot={slot.name} slotChildCount={srt.childCount} children={sb} cloneRoot={(cr != null ?
                    cr.name : \"null\")} rawCollect={(raw != null)} rawType={(raw != null ?
                    (int)raw.RuntimeCollectType : -1)} rawPoolManaged={(raw != null && raw.IsViewPoolManaged)}");
            }
        #else
            else
                UnityEngine.Debug.Log(
                    "[BlockRelaxCollect] Detach ok cell=({brd.x}, {brd.y})
                    type={(int)collect.RuntimeCollectType} go={collect.name}");
        #endif
        if (collect == null)
            continue;
        collect.gameObject.SetActive(true);
        collect.transform.SetParent(flyParent, true);
        collect.transform.SetAsLastSibling();
        _detachedPlacedCollectFlyScratch.Add(collect);
    }
    #if UNITY_EDITOR || DEVELOPMENT_BUILD
        UnityEngine.Debug.Log($"[BlockRelaxCollect] Detach end
```

方块解压馆软件 V1.0.0 源代码

```
queued={_detachedPlacedCollectFlyScratch.Count}");
#endif
}
    public UniTask FlyDetachedPlacedCollectsToObjectiveAsync(GameBlockRelaxModel model) =>
        FlyDetachedPlacedCollectsToObjectiveInternalAsync(model);
    private async UniTask
FlyDetachedPlacedCollectsToObjectiveInternalAsync(GameBlockRelaxModel model)
    {
        if (model == null || _detachedPlacedCollectFlyScratch.Count == 0)
        {
            #if UNITY_EDITOR || DEVELOPMENT_BUILD
                UnityEngine.Debug.Log(
                    $"[BlockRelaxCollect] Fly exit early: modelNull={model == null}
detachCount={_detachedPlacedCollectFlyScratch.Count}");
            #endif
            _detachedPlacedCollectFlyScratch.Clear();
            return;
        }
        var list = new
List<BlockRelaxItemCollectView>(_detachedPlacedCollectFlyScratch.Count);
        for (int i = 0; i < _detachedPlacedCollectFlyScratch.Count; i++)
        {
            var v = _detachedPlacedCollectFlyScratch[i];
            if (v != null)
                list.Add(v);
        }
        _detachedPlacedCollectFlyScratch.Clear();
        #if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log($"[BlockRelaxCollect] Fly begin listCount={list.Count}");
        #endif
        if (list.Count > 0)
        {
            var tasks = new UniTask[list.Count];
            for (int i = 0; i < list.Count; i++)
                tasks[i] = FlyOneDetachedCollectAsync(list[i], model);
            await UniTask.WhenAll(tasks);
        }
        #if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log("[BlockRelaxCollect] Fly end all awaited");
        #endif
    }
    private async UniTask FlyOneDetachedCollectAsync(BlockRelaxItemCollectView v,
GameBlockRelaxModel model)
    {
        if (v == null || model == null)
        {
            #if UNITY_EDITOR || DEVELOPMENT_BUILD
                UnityEngine.Debug.Log($"[BlockRelaxCollect] FlyOne skip: vNull={v == null}
modelNull={model == null}");
            #endif
            return;
        }
        var type = v.RuntimeCollectType;
```

方块解压馆软件 V1.0.0 源代码

```
        if (type == BlockRelaxCollectType.None || !v.IsViewPoolManaged)
        {
#if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log(
                $"[BlockRelaxCollect] FlyOne release early: type={{(int)type}}
poolManaged={{v.IsViewPoolManaged}} go={{v.name}}");
#endif
            ReleaseBlockRelaxItemCollectViewToPool(v);
            return;
        }
        if (!TryGetCollectObjectiveRowIconRect(type, out var targetRt) || targetRt == null)
        {
#if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log(
                $"[BlockRelaxCollect] FlyOne no row icon → register only type={{(int)type}}
go={{v.name}}");
#endif
            model.RegisterCollectObjectiveProgress(type, 1);
            RefreshCollectObjectiveRowCounts(model);
            ReleaseBlockRelaxItemCollectViewToPool(v);
            return;
        }
        var flyRt = GetCollectViewRectTransform(v);
        if (flyRt == null)
        {
#if UNITY_EDITOR || DEVELOPMENT_BUILD
            UnityEngine.Debug.Log($"[BlockRelaxCollect] FlyOne flyRt null go={{v.name}}");
#endif
            ReleaseBlockRelaxItemCollectViewToPool(v);
            return;
        }
#if UNITY_EDITOR || DEVELOPMENT_BUILD
        UnityEngine.Debug.Log(
            $"[BlockRelaxCollect] FlyOne animate type={{(int)type}} start={{flyRt.position}}
end={{targetRt.position}}");
#endif
        Vector3 start = flyRt.position;
        Vector3 end = targetRt.position;
        Quaternion startRot = flyRt.rotation;
        Quaternion endRot = targetRt.rotation;
        Vector3 startScale = flyRt.localScale;
        Vector3 endScale = startScale * CollectFlyEndScale;
        float elapsed = 0f;
        while (elapsed < CollectFlyDurationSeconds)
        {
            elapsed += Time.unscaledDeltaTime;
            float t = Mathf.Clamp01(elapsed / CollectFlyDurationSeconds);
            float e = t * t * (3f - 2f * t);
            flyRt.position = Vector3.LerpUnclamped(start, end, e);
            flyRt.rotation = Quaternion.SlerpUnclamped(startRot, endRot, e);
            flyRt.localScale = Vector3.LerpUnclamped(startScale, endScale, e);
            await UniTask.Yield(PlayerLoopTiming.Update);
        }
    }
```

方块解压馆软件 V1.0.0 源代码

```
#if UNITY_EDITOR || DEVELOPMENT_BUILD
    UnityEngine.Debug.Log($"[BlockRelaxCollect]    FlyOne    done    (before    pool)
type={{(int)type} go={{v.name}}");
#endif

    model.RegisterCollectObjectiveProgress(type, 1);
    RefreshCollectObjectiveRowCounts(model);
    ReleaseBlockRelaxItemCollectViewToPool(v);
}
}
}

using System;
using System.Collections.Generic;
using Cysharp.Threading.Tasks;
using FW.Base;
using FW.Unity;
using UnityEngine;
using UnityEngine.UI;
namespace View.UI
{
    public partial class GameBlockRelaxView
    {
        private const string BlockRelaxVfxGroup = "BlockRelax";
        private const string BlockRelaxEliminateSfxGroup = "UI";
        private const float EliminateTimeEpsilon = 1e-5f;
        public async UniTask ApplyEliminateLineClearStaggeredAsync(
            IReadOnlyList<Vector2Int> boardCells,
            int boardCols,
            int boardRows)
        {
            if (_gridSlots == null || boardCells == null || boardCells.Count == 0)
                return;
            var config = ConfigManager.Get<GameConfigData>(ConfigAddresses.GameConfig);
            float dt = config != null ? config.BlockRelaxEliminateCellStaggerSeconds : 0.04f;
            string sfx = config != null ? config.SfxBlockRelaxEliminate :
"SFX_BlockRelax_Eliminate";
            string vfx = config != null ? config.VfxBlockRelaxEliminate :
"VFX_BlockRelax_Eliminate";
            bool mergeSfx = config == null || config.BlockRelaxEliminateSfxMergeSameTimeStep;
            float batchMinVol = config != null ? config.BlockRelaxEliminateSfxBatchMinVolume :
0.22f;

            var set = new HashSet<Vector2Int>();
            for (int i = 0; i < boardCells.Count; i++)
                set.Add(boardCells[i]);
            var fullRows = new bool[boardRows];
            var fullCols = new bool[boardCols];
            for (int r = 0; r < boardRows; r++)
                fullRows[r] = IsRowFullForEliminationSet(set, r, boardCols);
            for (int c = 0; c < boardCols; c++)
                fullCols[c] = IsColFullForEliminationSet(set, c, boardRows);
            var scheduled = new List<(Vector2Int cell, float time)>(boardCells.Count);
            foreach (var p in set)
            {
                bool rowLine = fullRows[p.y];
```

方块解压馆软件 V1.0.0 源代码

```
        bool colLine = fullCols[p.x];
        float tRow = rowLine ? p.x * dt : float.MaxValue;
        float tCol = colLine ? p.y * dt : float.MaxValue;
        float t = Mathf.Min(tRow, tCol);
        if (t >= float.MaxValue * 0.5f)
            t = 0f;
        scheduled.Add((p, t));
    }
    scheduled.Sort(CompareScheduledCell);
    float tCursor = 0f;
    int idx = 0;
    while (idx < scheduled.Count)
    {
        float tNeed = scheduled[idx].time;
        float wait = tNeed - tCursor;
        if (wait > 1e-4f)
        {
            await UniTask.Delay(
                TimeSpan.FromSeconds(wait),
                DelayType.UnscaledDeltaTime);
            tCursor = tNeed;
        }
        int batchEnd = idx + 1;
        while (batchEnd < scheduled.Count
            && Mathf.Abs(scheduled[batchEnd].time - tNeed) < EliminateTimeEpsilon)
            batchEnd++;
        int batchCount = batchEnd - idx;
        if (mergeSfx && batchCount > 1)
        {
            float vol = Mathf.Clamp(1f / Mathf.Sqrt(batchCount), batchMinVol, 1f);
            for (int j = idx; j < batchEnd; j++)
            {
                var cell = scheduled[j].cell;
                PlayEliminateCellVfx(cell.x, cell.y, vfx);
                DestroyPlacedChildrenUnderSlot(cell.x, cell.y);
            }
            PlayEliminateSfx(sfx, vol);
        }
        else
        {
            for (int j = idx; j < batchEnd; j++)
            {
                var cell = scheduled[j].cell;
                PlayEliminateCellVfx(cell.x, cell.y, vfx);
                PlayEliminateSfx(sfx, 1f);
                DestroyPlacedChildrenUnderSlot(cell.x, cell.y);
            }
        }
        idx = batchEnd;
    }
}

private static int CompareScheduledCell((Vector2Int cell, float time) a, (Vector2Int
cell, float time) b)
```

方块解压馆软件 V1.0.0 源代码

```
{
    int c = a.time.CompareTo(b.time);
    if (c != 0)
        return c;
    c = a.cell.x.CompareTo(b.cell.x);
    if (c != 0)
        return c;
    return a.cell.y.CompareTo(b.cell.y);
}
private static bool IsRowFullForEliminationSet(HashSet<Vector2Int> set, int row, int
cols)
{
    for (int c = 0; c < cols; c++)
    {
        if (!set.Contains(new Vector2Int(c, row)))
            return false;
    }
    return true;
}
private static bool IsColFullForEliminationSet(HashSet<Vector2Int> set, int col, int
rows)
{
    for (int r = 0; r < rows; r++)
    {
        if (!set.Contains(new Vector2Int(col, r)))
            return false;
    }
    return true;
}
public void PlayReviveExploreFeedback(IReadOnlyList<Vector2Int> boardCells)
{
    if (_gridSlots == null || boardCells == null || boardCells.Count == 0)
        return;
    var config = ConfigManager.Get<GameConfigData>(ConfigAddresses.GameConfig);
    string sfx = config != null ? config.SfxBlockRelaxExplore :
"SFX_BlockRelax_Explore";
    string vfx = config != null ? config.VfxBlockRelaxEliminate :
"VFX_BlockRelax_Eliminate";
    for (int i = 0; i < boardCells.Count; i++)
        PlayEliminateCellVfx(boardCells[i].x, boardCells[i].y, vfx);
    if (string.IsNullOrEmpty(sfx) || Game.Audio == null)
        return;
    Game.Audio.Play(sfx, BlockRelaxEliminateSfxGroup);
}
private void PlayEliminateCellVfx(int col, int row, string vfxAddress)
{
    if (string.IsNullOrEmpty(vfxAddress) || Game.VFX == null)
        return;
    if (!TryGetSlotWorldCenter(col, row, out var worldPos))
        return;
    bool hasCellSprite = TryGetPlacedCellSpriteFromBoardSlot(col, row, out Sprite
cellSprite)
        && cellSprite != null && cellSprite.texture != null;
```

```
        if (hasCellSprite)
        {
            Sprite skin = cellSprite;
            Game.VFX.Play(
                vfxAddress,
                BlockRelaxVfxGroup,
                worldPos,
                Quaternion.identity,
                null,
                Vector3.one,
                go => ApplySpriteToEliminateVfxInstance(go.transform, skin));
        }
        else
        {
            Game.VFX.Play(vfxAddress, BlockRelaxVfxGroup, worldPos, Quaternion.identity,
            null, Vector3.one);
        }
    }
    private bool TryGetPlacedCellSpriteFromBoardSlot(int col, int row, out Sprite sprite)
    {
        sprite = null;
        var slot = GetSlot(col, row);
        if (slot == null)
            return false;
        var rt = slot.rectTransform;
        for (int i = 0; i < rt.childCount; i++)
        {
            var child = rt.GetChild(i);
            var itemView = child.GetComponentInChildren<BlockRelaxItemView>(true);
            if (itemView != null && itemView.TryGetIconSprite(out sprite) && sprite != null)
                return true;
            var img = child.GetComponentInChildren<Image>(true);
            if (img != null && img.sprite != null)
            {
                sprite = img.sprite;
                return true;
            }
        }
        return false;
    }
    private static void ApplySpriteToEliminateVfxInstance(Transform vfxRoot, Sprite sprite)
    {
        if (vfxRoot == null || sprite == null || sprite.texture == null)
            return;
        var renderers = vfxRoot.GetComponentsInChildren<ParticleSystemRenderer>(true);
        if (renderers == null || renderers.Length == 0)
            return;
        Vector4 st = MainTexScaleOffsetFromSprite(sprite);
        var block = new MaterialPropertyBlock();
        for (int i = 0; i < renderers.Length; i++)
        {
            var r = renderers[i];
            if (r == null)
```



```
        continue;
        r.GetPropertyBlock(block);
        block.SetTexture("_MainTex", sprite.texture);
        block.SetVector("_MainTex_ST", st);
        block.SetTexture("_BaseMap", sprite.texture);
        block.SetVector("_BaseMap_ST", st);
        r.SetPropertyBlock(block);
    }
}

private static Vector4 MainTexScaleOffsetFromSprite(Sprite sprite)
{
    Texture2D tex = sprite.texture;
    Rect tr = sprite.textureRect;
    float tw = tex.width;
    float th = tex.height;
    if (tw <= 1e-5f || th <= 1e-5f)
        return new Vector4(1f, 1f, 0f, 0f);
    return new Vector4(tr.width / tw, tr.height / th, tr.x / tw, tr.y / th);
}

private static void PlayEliminateSfx(string sfxAddress, float volume)
{
    if (string.IsNullOrEmpty(sfxAddress) || Game.Audio == null)
        return;
    volume = Mathf.Clamp01(volume);
    if (volume >= 0.999f)
        Game.Audio.Play(sfxAddress, BlockRelaxEliminateSfxGroup);
    else
        Game.Audio.PlayWithPitch(sfxAddress, BlockRelaxEliminateSfxGroup, volume,
1f);
}

private bool TryGetSlotWorldCenter(int col, int row, out Vector3 worldPos)
{
    worldPos = default;
    var slot = GetSlot(col, row);
    if (slot == null)
        return false;
    var rt = slot.rectTransform;
    var corners = new Vector3[4];
    rt.GetWorldCorners(corners);
    worldPos = (corners[0] + corners[2]) * 0.5f;
    return true;
}
}

using System;
using System.Collections.Generic;
using FW.Base;
using Model.UI;
using UnityEngine;
using UnityEngine.UI;
namespace View.UI
{
    public partial class GameBlockRelaxView
```

```
{
    public const int BoardCols = 8;
    public const int BoardRows = 8;
    public const int SelectTraySlotCount = 3;
    private Image[] _selectTraySlotsCached;
    private Image[,] _gridSlots;
    public Image[,] GridSlots => _gridSlots;
    private Vector2 _boardGridOriginLocal;
    private Vector2 _boardGridDeltaColLocal;
    private Vector2 _boardGridDeltaRowLocal;
    private bool _boardGridMetricsValid;
    private Rect[,] _slotBoundsBoardLocal;
    private readonly List<BlockRelaxSelectPiecePoolEntry> _pickThreeEntriesScratch = new
List<BlockRelaxSelectPiecePoolEntry>(3);
    private BlockRelaxSelectPiecePool _lastSelectPiecePoolForAnchor;
    private readonly List<Vector2Int> _debugShapeCellsScratch = new List<Vector2Int>(16);
    protected override void OnInit()
    {
        base.OnInit();
        BuildBoard();
        EnsureSelectTraySlotsCached();
        if (TextCombo != null)
            TextCombo.gameObject.SetActive(false);
        if (TextEliminationTier != null)
            TextEliminationTier.gameObject.SetActive(false);
        InitScoreTargetView();
        PreloadBlockRelaxItemCollectPrefabIfNeeded();
    }
    private void EnsureSelectTraySlotsCached()
    {
        if (_selectTraySlotsCached != null)
            return;
        _selectTraySlotsCached = new Image[SelectTraySlotCount];
        _selectTraySlotsCached[0] = SelectItem_0;
        _selectTraySlotsCached[1] = SelectItem_1;
        _selectTraySlotsCached[2] = SelectItem_2;
    }
    public ReadOnlySpan<Image> GetSelectTraySlots()
    {
        EnsureSelectTraySlotsCached();
        return _selectTraySlotsCached;
    }
    public void RefreshSelectPieces(BlockRelaxSelectPiecePool pool, GameBlockRelaxModel
model)
    {
        _lastSelectPiecePoolForAnchor = pool;
        foreach (var slot in GetSelectTraySlots())
        {
            if (slot != null)
                ClearSelectSpawns(slot.rectTransform);
        }
        if (pool == null)
        {

```

方块解压馆软件 V1.0.0 源代码

```
        for (int i = 0; i < SelectTraySlotCount; i++)
            EnsureSelectedItemDrag(_selectTraySlotsCached[i], null, i);
        return;
    }
    if (model?.BoardOccupied == null)
    {
        Debug.LogWarning("[GameBlockRelaxView] RefreshSelectPieces : model 或
BoardOccupied 为空, 无法带盘抽块, 已跳过 PickThreeEntries。");
        for (int i = 0; i < SelectTraySlotCount; i++)
            EnsureSelectedItemDrag(_selectTraySlotsCached[i], null, i);
        return;
    }
    float difficulty01 = model.SelectPieceDifficulty01;
    pool.PickThreeEntries(_pickThreeEntriesScratch, difficulty01, model.BoardOccupied,
BoardCols, BoardRows);
    var picks = _pickThreeEntriesScratch;
    for (int i = 0; i < SelectTraySlotCount; i++)
    {
        var slot = _selectTraySlotsCached[i];
        if (slot == null)
            continue;
        if (i >= picks.Count)
        {
            EnsureSelectedItemDrag(slot, null, i);
            continue;
        }
        var entry = picks[i];
        var piecePrefab = pool?.GetPrefabForEntry(entry);
        if (piecePrefab == null)
        {
            EnsureSelectedItemDrag(slot, null, i);
            continue;
        }
        SpawnSelectPieceOnSlot(slot, pool, entry, i, model);
    }
}

private void EnsureSelectedItemDrag(Image slot, RectTransform pieceRoot, int slotIndex)
{
    if (slot == null)
        return;
    slot.raycastTarget = true;
    var drag = slot.GetComponent<BlockRelaxSelectItemDrag>();
    if (drag == null)
        drag = slot.gameObject.AddComponent<BlockRelaxSelectItemDrag>();
    drag.Initialize(this, slotIndex);
    drag.BindPiece(pieceRoot);
}

private void ClearSelectSpawns(RectTransform parent)
{
    for (int c = parent.childCount - 1; c >= 0; c--)
    {
        var t = parent.GetChild(c);
        if (!t.name.StartsWith("SelectPiece_"))
    }
```

方块解压馆软件 V1.0.0 源代码

```
        continue;
    if (t is RectTransform pieceRt)
        ReleaseRuntimeCollectsUnderPieceRoot(pieceRt);
    UnityEngine.Object.DestroyImmediate(t.gameObject);
}
}
public void BuildBoard()
{
    if (ImageSlotBack == null || ImageSlot == null)
    {
        Debug.LogWarning("[GameBlockRelaxView] ImageSlotBack 或 ImageSlot 未绑定。");
        return;
    }
    var boardRt = ImageSlotBack.rectTransform;
    ClearGeneratedSlots(boardRt);
    var templateGo = ImageSlot.gameObject;
    templateGo.SetActive(false);
    _gridSlots = new Image[BoardCols, BoardRows];
    for (int row = 0; row < BoardRows; row++)
    {
        for (int col = 0; col < BoardCols; col++)
        {
            var inst = UnityEngine.Object.Instantiate(templateGo, boardRt);
            inst.name = $"ImageSlot_{col}_{row}";
            inst.SetActive(true);
            _gridSlots[col, row] = inst.GetComponent<Image>();
        }
    }
    LayoutRebuilder.ForceRebuildLayoutImmediate(boardRt);
    NormalizeGridSlotRowsSoRowZeroIsBottom(boardRt);
    RefreshBoardGridGeometry();
    EnsureBoardPlacementPreviewPool();
    RefreshBlockRelaxBoardDebugCellLabels();
}
private void NormalizeGridSlotRowsSoRowZeroIsBottom(RectTransform boardRt)
{
    if (_gridSlots == null || BoardRows < 2)
        return;
    float yRow0 = SlotBottomLeftBoardLocalY(boardRt, _gridSlots[0, 0].rectTransform);
    float yRow1 = SlotBottomLeftBoardLocalY(boardRt, _gridSlots[0, 1].rectTransform);
    if (yRow1 > yRow0 - 1e-4f)
        return;
    for (int c = 0; c < BoardCols; c++)
    {
        for (int r = 0; r < BoardRows / 2; r++)
        {
            int r2 = BoardRows - 1 - r;
            (_gridSlots[c, r], _gridSlots[c, r2]) = (_gridSlots[c, r2], _gridSlots[c,
r]);
        }
    }
    for (int row = 0; row < BoardRows; row++)
    {
```

方块解压馆软件 V1.0.0 源代码

```
        for (int col = 0; col < BoardCols; col++)
        {
            var img = _gridSlots[col, row];
            if (img != null)
                img.gameObject.name = $"ImageSlot_{col}_{row}";
        }
    }
}

private float SlotBottomLeftBoardLocalY(RectTransform boardRt, RectTransform slotRt)
{
    slotRt.GetWorldCorners(_workCorners);
    return boardRt.InverseTransformPoint(_workCorners[0]).y;
}

private void RefreshBoardGridGeometry()
{
    _boardGridMetricsValid = false;
    _slotBoundsBoardLocal = null;
    if (_gridSlots == null || ImageSlotBack == null)
        return;
    var boardRt = ImageSlotBack.rectTransform;
    Rect br = boardRt.rect;
    int cols = _gridSlots.GetLength(0);
    int rows = _gridSlots.GetLength(1);
    _slotBoundsBoardLocal = new Rect[cols, rows];
    for (int r = 0; r < rows; r++)
    {
        for (int c = 0; c < cols; c++)
        {
            var slotRt = _gridSlots[c, r].rectTransform;
            _slotBoundsBoardLocal[c, r] = ComputeSlotBoardLocalBounds(boardRt, slotRt,
_workCorners);
        }
    }
}

Vector2 CellBottomLeftLocal(int col, int row)
{
    var rt = _gridSlots[col, row].rectTransform;
    rt.GetWorldCorners(_workCorners);
    return (Vector2)boardRt.InverseTransformPoint(_workCorners[0]);
}

_boardGridOriginLocal = CellBottomLeftLocal(0, 0);
_boardGridDeltaColLocal = cols > 1
    ? CellBottomLeftLocal(1, 0) - _boardGridOriginLocal
    : new Vector2(br.width / Mathf.Max(1, cols), 0f);
_boardGridDeltaRowLocal = rows > 1
    ? CellBottomLeftLocal(0, 1) - _boardGridOriginLocal
    : new Vector2(0f, br.height / Mathf.Max(1, rows));
if (_boardGridDeltaColLocal.sqrMagnitude < 1e-12f &&
_boardGridDeltaRowLocal.sqrMagnitude < 1e-12f)
    return;
_boardGridMetricsValid = true;
#if UNITY_EDITOR || DEVELOPMENT_BUILD
    Log.Debug(
        $"[BlockRelaxView] RefreshBoardGridGeometry cols={cols} rows={rows} " +
```

方块解压馆软件 V1.0.0 源代码

```
        $"originLocal={_boardGridOriginLocal}      deltaCol={_boardGridDeltaColLocal}
deltaRow={_boardGridDeltaRowLocal} " +
        $"|deltaCol|={_boardGridDeltaColLocal.magnitude:F4}
|deltaRow|={_boardGridDeltaRowLocal.magnitude:F4}");
#endif
    }

    private static Rect ComputeSlotBoardLocalBounds(RectTransform boardRt, RectTransform
slotRt, Vector3[] cornersBuf)
    {
        slotRt.GetWorldCorners(cornersBuf);
        float minX = float.PositiveInfinity, minY = float.PositiveInfinity;
        float maxX = float.NegativeInfinity, maxY = float.NegativeInfinity;
        for (int i = 0; i < 4; i++)
        {
            Vector2 bl = boardRt.InverseTransformPoint(cornersBuf[i]);
            if (bl.x < minX) minX = bl.x;
            if (bl.y < minY) minY = bl.y;
            if (bl.x > maxX) maxX = bl.x;
            if (bl.y > maxY) maxY = bl.y;
        }
        return Rect.MinMaxRect(minX, minY, maxX, maxY);
    }

    private static float SqrDistancePointToRect(Vector2 p, Rect r)
    {
        float cx = Mathf.Clamp(p.x, r.xMin, r.xMax);
        float cy = Mathf.Clamp(p.y, r.yMin, r.yMax);
        float dx = p.x - cx;
        float dy = p.y - cy;
        return dx * dx + dy * dy;
    }

    private bool TryGetCombinedBoardSlotsBoundsLocal(out Rect union)
    {
        union = default;
        if (_slotBoundsBoardLocal == null)
            return false;
        float minX = float.PositiveInfinity, minY = float.PositiveInfinity;
        float maxX = float.NegativeInfinity, maxY = float.NegativeInfinity;
        for (int r = 0; r < BoardRows; r++)
        {
            for (int c = 0; c < BoardCols; c++)
            {
                var cell = _slotBoundsBoardLocal[c, r];
                if (cell.xMin < minX)
                    minX = cell.xMin;
                if (cell.yMin < minY)
                    minY = cell.yMin;
                if (cell.xMax > maxX)
                    maxX = cell.xMax;
                if (cell.yMax > maxY)
                    maxY = cell.yMax;
            }
        }
        if (!(minX < maxX && minY < maxY))
            return false;
        union = Rect.MinMaxRect(minX, minY, maxX, maxY);
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        return false;
        union = Rect.MinMaxRect(minX, minY, maxX, maxY);
        return true;
    }
    private bool TryBoardLocalPointToAnchorSeed(Vector3 anchorBoardLocal, out int seedCol,
out int seedRow)
    {
        seedCol = seedRow = 0;
        if (_slotBoundsBoardLocal == null || ImageSlotBack == null || _gridSlots == null)
            return false;
        var p = new Vector2(anchorBoardLocal.x, anchorBoardLocal.y);
        for (int r = 0; r < BoardRows; r++)
        {
            for (int c = 0; c < BoardCols; c++)
            {
                if (_slotBoundsBoardLocal[c, r].Contains(p))
                {
                    seedCol = c;
                    seedRow = r;
                    return true;
                }
            }
        }
        if (!TryGetCombinedBoardSlotsBoundsLocal(out var boardAabb))
            return false;
        if (boardAabb.Contains(p))
        {
            PickNearestCellByRectDistance(p, out seedCol, out seedRow);
            return true;
        }
        if (TryVirtualGridFloorSeedFromBoardLocalPoint(p, out seedCol, out seedRow))
            return true;
        PickNearestCellByRectDistance(p, out seedCol, out seedRow);
        return true;
    }
    private void PickNearestCellByRectDistance(Vector2 p, out int bestC, out int bestR)
    {
        float best = float.PositiveInfinity;
        bestC = 0;
        bestR = 0;
        for (int r = 0; r < BoardRows; r++)
        {
            for (int c = 0; c < BoardCols; c++)
            {
                float d = SqrDistancePointToRect(p, _slotBoundsBoardLocal[c, r]);
                if (d < best || (Mathf.Approximately(d, best) && (r < bestR || (r == bestR
&& c < bestC))))
                {
                    best = d;
                    bestC = c;
                    bestR = r;
                }
            }
        }
    }
```

```

    }
}

private bool TryVirtualGridFloorSeedFromBoardLocalPoint(Vector2 p, out int seedCol, out
int seedRow)
{
    seedCol = seedRow = 0;
    if (!_boardGridMetricsValid)
        return false;
    Vector2 u = _boardGridDeltaColLocal;
    Vector2 v = _boardGridDeltaRowLocal;
    Vector2 rel = p - _boardGridOriginLocal;
    float det = u.x * v.y - u.y * v.x;
    if (Mathf.Abs(det) < 1e-10f)
    {
        if (Mathf.Abs(u.x) > 1e-6f && Mathf.Abs(v.y) > 1e-6f)
        {
            seedCol = Mathf.FloorToInt(rel.x / u.x);
            seedRow = Mathf.FloorToInt(rel.y / v.y);
            return true;
        }
        return false;
    }
    float colF = (rel.x * v.y - rel.y * v.x) / det;
    float rowF = (u.x * rel.y - u.y * rel.x) / det;
    seedCol = Mathf.FloorToInt(colF);
    seedRow = Mathf.FloorToInt(rowF);
    return true;
}

private bool TryBoardLocalPointToNearestCell(Vector3 anchorBoardLocal, out int col, out
int row)
{
    col = row = 0;
    if (_slotBoundsBoardLocal == null || ImageSlotBack == null || _gridSlots == null)
        return false;
    var p = new Vector2(anchorBoardLocal.x, anchorBoardLocal.y);
    for (int r = 0; r < BoardRows; r++)
    {
        for (int c = 0; c < BoardCols; c++)
        {
            if (_slotBoundsBoardLocal[c, r].Contains(p))
            {
                col = c;
                row = r;
                return true;
            }
        }
    }
    if (!TryGetCombinedBoardSlotsBoundsLocal(out _))
        return false;
    PickNearestCellByRectDistance(p, out col, out row);
    return true;
}

private static void ClearGeneratedSlots(RectTransform boardRt)

```


方块解压馆软件 V1.0.0 源代码

```
{
    for (int i = boardRt.childCount - 1; i >= 0; i--)
    {
        var go = boardRt.GetChild(i).gameObject;
        if (go.name.StartsWith("ImageSlot_"))
            UnityEngine.Object.Destroy(go);
    }
}

public Image GetSlot(int col, int row)
{
    if (_gridSlots == null || col < 0 || col >= BoardCols || row < 0 || row >= BoardRows)
        return null;
    return _gridSlots[col, row];
}

public bool TryScreenPointToBoardCell(Vector2 screenPoint, Camera uiCamera, out int col,
out int row)
{
    col = row = 0;
    if (ImageSlotBack == null)
        return false;
    var boardRt = ImageSlotBack.rectTransform;
    if (!RectTransformUtility.ScreenPointToLocalPointInRectangle(boardRt, screenPoint,
uiCamera, out var local))
        return false;
    Rect r = boardRt.rect;
    if (local.x < r.xMin || local.x > r.xMax || local.y < r.yMin || local.y > r.yMax)
        return false;
    float u = (local.x - r.xMin) / r.width;
    float v = (local.y - r.yMin) / r.height;
    col = Mathf.Clamp(Mathf.FloorToInt(u * BoardCols), 0, BoardCols - 1);
    row = Mathf.Clamp(Mathf.FloorToInt(v * BoardRows), 0, BoardRows - 1);
    return true;
}

public bool TryScreenPointToBoardCellBySlotBounds(Vector2 screenPoint, Camera uiCamera,
out int col, out int row)
{
    col = row = 0;
    if (_gridSlots == null)
        return false;
    for (int ry = 0; ry < BoardRows; ry++)
    {
        for (int cx = 0; cx < BoardCols; cx++)
        {
            var img = _gridSlots[cx, ry];
            if (img == null)
                continue;
            var rt = img.rectTransform;
            if (RectTransformUtility.RectangleContainsScreenPoint(rt, screenPoint,
uiCamera))
            {
                col = cx;
                row = ry;
                return true;
            }
        }
    }
}
```

```
        }
    }
}
return false;
}
private void SpawnSelectPieceOnSlot(
    Image slot,
    BlockRelaxSelectPiecePool pool,
    BlockRelaxSelectPiecePoolEntry entry,
    int slotIndex,
    GameBlockRelaxModel model)
{
    var prefab = pool != null ? pool.GetPrefabForEntry(entry) : null;
    if (prefab == null)
        return;
    var parent = slot.rectTransform;
    var go = UnityEngine.Object.Instantiate(prefab, parent);
    go.name = $"SelectPiece_{slotIndex}_{prefab.name}";
    go.SetActive(true);
    RectTransform pieceRt = null;
    if (go.transform is RectTransform rt)
    {
        pieceRt = rt;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.pivot = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = Vector2.zero;
        rt.localRotation = Quaternion.identity;
        rt.localScale = entry.PreviewScale;
    }
    else
    {
        go.transform.localPosition = Vector3.zero;
        go.transform.localRotation = Quaternion.identity;
        go.transform.localScale = entry.PreviewScale;
    }
    if (pieceRt != null)
    {
        var itemType = entry.ItemType == BlockRelaxItemType.None ?
BlockRelaxItemType.JellyBlue : entry.ItemType;
        TryApplyItemSkinSpritesUnderPiece(pieceRt, model, itemType);
        TrySpawnCollectOnSelectPiece(pieceRt, entry, model);
    }
    EnsureSelectItemDrag(slot, pieceRt, slotIndex);
}
public void SetLevelDisplay(BlockRelaxLevelEntry entry, int zeroBasedLevelIndex)
{
    if (TextLevel == null)
        return;
    if (entry != null && !string.IsNullOrEmpty(entry.DisplayName))
    {
        TextLevel.text = entry.DisplayName.Trim();
        return;
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        if (entry != null && entry.LevelId > 0)
        {
            TextLevel.text = $"关卡 {entry.LevelId}";
            return;
        }
        TextLevel.text = $"关卡 {zeroBasedLevelIndex + 1}";
    }

    public void SetScoreDisplay(int currentScore, BlockRelaxLevelEntry entry,
GameBlockRelaxModel model)
    {
        bool showCollect = entry != null && entry.CompletionMode ==
BlockRelaxLevelCompletionMode.ItemObjectives;
        bool showScore =
            !showCollect
            && (
                entry == null
                || entry.CompletionMode != BlockRelaxLevelCompletionMode.ItemObjectives
                || entry.TargetScore > 0);
        bool showTargetProgress = showScore && entry != null && entry.TargetScore > 0;
        if (PanelCollect != null)
            PanelCollect.gameObject.SetActive(showCollect);
        if (_panelScoreRoot != null)
            _panelScoreRoot.gameObject.SetActive(showScore);
        if (PanelScoreTarget != null)
            PanelScoreTarget.gameObject.SetActive(showScore);
        if (_scoreTargetView != null && showScore)
            _scoreTargetView.SetScoreTargetDisplay(showTargetProgress, currentScore,
entry);
        if (TextScore != null)
            TextScore.text = currentScore.ToString();
        if (showCollect)
            RefreshCollectObjectiveRows(model, entry);
        else
            ClearCollectStageRuntimeRows();
    }
}

}

using System.Collections.Generic;
using Model.UI;
using UnityEngine;
namespace View.UI
{
    public partial class GameBlockRelaxView
    {
        private struct CollectStageRowSpec
        {
            public BlockRelaxCollectType CollectType;
            public int RequiredCount;
        }
        private GameBlockRelaxScoreTargetView _scoreTargetView;
        private Transform _panelScoreRoot;
        private readonly List<GameObject> _collectStageRowInstances = new List<GameObject>(8);
        private readonly List<CollectStageRowSpec> _collectStageRowSpecs = new
```

方块解压馆软件 V1.0.0 源代码

```
List<CollectStageRowSpec>(8);
    private void InitScoreTargetView()
    {
        _panelScoreRoot = TryResolvePanelScoreRootTransform();
        if (PanelScoreTarget == null)
            return;
        _scoreTargetView
PanelScoreTarget.GetComponent<GameBlockRelaxScoreTargetView>();
        if (_scoreTargetView == null)
            _scoreTargetView
PanelScoreTarget.GetComponentInChildren<GameBlockRelaxScoreTargetView>(true);
    }
    private Transform TryResolvePanelScoreRootTransform()
    {
        if (PanelCollect != null && PanelCollect.parent != null)
        {
            Transform p = PanelCollect.parent;
            for (int i = 0; i < p.childCount; i++)
            {
                Transform c = p.GetChild(i);
                if (c.name == "PanelScore")
                    return c;
            }
        }
        if (PanelScoreTarget != null
            && PanelScoreTarget.parent != null
            && PanelScoreTarget.parent.parent != null
            && PanelScoreTarget.parent.parent.name == "PanelScore")
            return PanelScoreTarget.parent.parent;
        return null;
    }
    private void ClearCollectStageRuntimeRows()
    {
        for (int i = 0; i < _collectStageRowInstances.Count; i++)
        {
            var go = _collectStageRowInstances[i];
            if (go != null)
                Object.Destroy(go);
        }
        _collectStageRowInstances.Clear();
        _collectStageRowSpecs.Clear();
    }
    private void RefreshCollectObjectiveRows(GameBlockRelaxModel model,
BlockRelaxLevelEntry entry)
    {
        ClearCollectStageRuntimeRows();
        if (model == null || entry == null || StageCollectTemplate == null ||
entry.CompletionConditions == null)
            return;
        var parent = StageCollectTemplate.transform.parent;
        if (parent == null)
            return;
        for (int i = 0; i < entry.CompletionConditions.Count; i++)
```

方块解压馆软件 V1.0.0 源代码

```
{
    var cond = entry.CompletionConditions[i];
    if (cond == null || cond.CollectType == BlockRelaxCollectType.None ||
cond.RequiredCount <= 0)
        continue;
    var go = Object.Instantiate(StageCollectTemplate.gameObject, parent, false);
    go.name = $"CollectStage_{(int)cond.CollectType}_{i}";
    go.SetActive(true);
    _collectStageRowInstances.Add(go);
    _collectStageRowSpecs.Add(new CollectStageRowSpec
    {
        CollectType = cond.CollectType,
        RequiredCount = cond.RequiredCount,
    });
    if (model.TryGetCollectSprite(cond.CollectType, out var sp) && sp != null)
    {
        var iconTarget = go.GetComponent<IBlockRelaxSetIconSprite>();
        if (iconTarget == null)
            iconTarget =
go.GetComponentInChildren<IBlockRelaxSetIconSprite>(true);
        iconTarget?.SetIconSprite(sp);
    }
    var rowView = go.GetComponent<GameBlockRelaxItemTargetView>();
    if (rowView == null)
        rowView = go.GetComponentInChildren<GameBlockRelaxItemTargetView>(true);
    if (rowView != null)
    {
        int cur = model.GetCollectObjectiveProgress(cond.CollectType);
        rowView.SetCount(cur, cond.RequiredCount);
    }
}

}

public bool TryGetCollectObjectiveRowIconRect(BlockRelaxCollectType collectType, out
RectTransform iconRt)
{
    iconRt = null;
    if (collectType == BlockRelaxCollectType.None)
        return false;
    for (int i = 0; i < _collectStageRowInstances.Count && i <
_collectStageRowSpecs.Count; i++)
    {
        if (_collectStageRowSpecs[i].CollectType != collectType)
            continue;
        var go = _collectStageRowInstances[i];
        if (go == null)
            continue;
        var rowView = go.GetComponent<GameBlockRelaxItemTargetView>();
        if (rowView == null)
            rowView = go.GetComponentInChildren<GameBlockRelaxItemTargetView>(true);
        return rowView != null && rowView.TryGetIconRectTransform(out iconRt);
    }
    return false;
}
```

方块解压馆软件 V1.0.0 源代码

```
public void RefreshCollectObjectiveRowCounts(GameBlockRelaxModel model)
{
    if (model == null || _collectStageRowInstances.Count == 0 ||
        _collectStageRowSpecs.Count != _collectStageRowInstances.Count)
        return;
    for (int i = 0; i < _collectStageRowInstances.Count; i++)
    {
        var go = _collectStageRowInstances[i];
        if (go == null)
            continue;
        var rowView = go.GetComponent<GameBlockRelaxItemTargetView>();
        if (rowView == null)
            rowView = go.GetComponentInChildren<GameBlockRelaxItemTargetView>(true);
        if (rowView == null)
            continue;
        var spec = _collectStageRowSpecs[i];
        int cur = model.GetCollectObjectiveProgress(spec.CollectType);
        rowView.SetCount(cur, spec.RequiredCount);
    }
}

}

}

using FW.Unity;
using TPro;
using UnityEngine;
using UnityEngine.Events;
using UnityEngine.UI;
namespace View.UI
{
    public partial class GameBlockRelaxEndView
    {
        private const string ReviveRemainingCountGreenHex = "#22C55E";
        public void ResetEndPanelsBeforeConfigure()
        {
            if (PanelRetry != null)
                PanelRetry.gameObject.SetActive(false);
            if (PanelRevive != null)
                PanelRevive.gameObject.SetActive(false);
            if (PanelEnd != null)
                PanelEnd.gameObject.SetActive(false);
            if (ButtonNext != null)
            {
                ButtonNext.gameObject.SetActive(false);
                ButtonNext.onClick.RemoveAllListeners();
            }
            if (ButtonBack != null)
            {
                ButtonBack.gameObject.SetActive(false);
                ButtonBack.onClick.RemoveAllListeners();
            }
            if (ButtonClose != null)
                ButtonClose.onClick.RemoveAllListeners();
            if (ButtonRetryClose != null)
```

方块解压馆软件 V1.0.0 源代码

```
        ButtonRetryClose.onClick.RemoveAllListeners();
    if (ButtonRevive != null)
    {
        ButtonRevive.onClick.RemoveAllListeners();
        ButtonRevive.interactable = true;
    }
    ClearReviveLevelGoalProgressUi();
    ClearRetryLevelGoalProgressUi();
    if (ButtonRetry != null)
        ButtonRetry.onClick.RemoveAllListeners();
}

public void SetReviveRemainingCount(int count)
{
    if (TextLeftNum == null)
        return;
    TextLeftNum.richText = true;
    TextLeftNum.text =
        $"剩余<color={ReviveRemainingCountGreenHex}>{count}</color>次";
}

public void SetRoundScoreText(int score)
{
    if (TextScore == null)
        return;
    TextScore.text = score.ToString();
}

public void ConfigureWinFlow(UnityAction onNextLevel, bool showNextLevelButton,
UnityAction onReturnToStart)
{
    if (PanelRetry != null)
        PanelRetry.gameObject.SetActive(false);
    if (PanelRevive != null)
        PanelRevive.gameObject.SetActive(false);
    if (PanelEnd != null)
        PanelEnd.gameObject.SetActive(true);
    if (ButtonBack != null)
    {
        ButtonBack.gameObject.SetActive(true);
        ButtonBack.onClick.RemoveAllListeners();
        ButtonBack.onClick.AddListener(() =>
        {
            ButtonClickSound.Play();
            onReturnToStart?.Invoke();
        });
    }
    if (ButtonNext != null)
    {
        ButtonNext.gameObject.SetActive(showNextLevelButton);
        ButtonNext.onClick.RemoveAllListeners();
        if (showNextLevelButton)
        {
            ButtonNext.onClick.AddListener(() =>
            {
                ButtonClickSound.Play();
            });
        }
    }
}
```

```

        onNextLevel?.Invoke();
    });
}
}
}
public void ConfigureLoseFlow(UnityAction onReturnStart)
{
    if (PanelRetry != null)
        PanelRetry.gameObject.SetActive(false);
    if (PanelRevive != null)
        PanelRevive.gameObject.SetActive(false);
    if (PanelEnd != null)
        PanelEnd.gameObject.SetActive(false);
    if (ButtonNext != null)
    {
        ButtonNext.gameObject.SetActive(false);
        ButtonNext.onClick.RemoveAllListeners();
    }
    if (ButtonBack != null)
    {
        ButtonBack.gameObject.SetActive(true);
        ButtonBack.onClick.RemoveAllListeners();
        ButtonBack.onClick.AddListener(() =>
        {
            ButtonClickSound.Play();
            onReturnStart?.Invoke();
        });
    }
}
public void ConfigureReviveOfferFlow(UnityAction onCloseReturnStart, UnityAction
onReviveClick)
{
    if (PanelRetry != null)
        PanelRetry.gameObject.SetActive(false);
    if (PanelRevive != null)
        PanelRevive.gameObject.SetActive(true);
    if (PanelEnd != null)
        PanelEnd.gameObject.SetActive(false);
    if (ButtonBack != null)
    {
        ButtonBack.gameObject.SetActive(false);
        ButtonBack.onClick.RemoveAllListeners();
    }
    if (ButtonNext != null)
    {
        ButtonNext.gameObject.SetActive(false);
        ButtonNext.onClick.RemoveAllListeners();
    }
    if (ButtonClose != null)
    {
        ButtonClose.onClick.RemoveAllListeners();
        ButtonClose.onClick.AddListener(() =>
        {

```



```
        ButtonClickSound.Play();
        onCloseReturnStart?.Invoke();
    });
}
if (ButtonRevive != null)
{
    ButtonRevive.onClick.RemoveAllListeners();
    ButtonRevive.onClick.AddListener(() =>
    {
        ButtonClickSound.Play();
        onReviveClick?.Invoke();
    });
}
}

public void ConfigureRetryFlow(UnityAction onReturnStart)
{
    if (PanelRevive != null)
        PanelRevive.gameObject.SetActive(false);
    if (PanelRetry != null)
        PanelRetry.gameObject.SetActive(true);
    if (PanelEnd != null)
        PanelEnd.gameObject.SetActive(false);
    if (ButtonNext != null)
    {
        ButtonNext.gameObject.SetActive(false);
        ButtonNext.onClick.RemoveAllListeners();
    }
    if (ButtonBack != null)
    {
        ButtonBack.gameObject.SetActive(false);
        ButtonBack.onClick.RemoveAllListeners();
    }
    if (ButtonRetryClose != null)
    {
        ButtonRetryClose.onClick.RemoveAllListeners();
        ButtonRetryClose.onClick.AddListener(() =>
        {
            ButtonClickSound.Play();
            onReturnStart?.Invoke();
        });
    }
    if (ButtonRetry != null)
    {
        ButtonRetry.onClick.RemoveAllListeners();
        ButtonRetry.onClick.AddListener(() =>
        {
            ButtonClickSound.Play();
            onReturnStart?.Invoke();
        });
    }
}

public void SetReviveButtonInteractable(bool interactable)
{

```

方块解压馆软件 V1.0.0 源代码

```
        if (ButtonRevive != null)
            ButtonRevive.interactable = interactable;
    }
    public void SetReviveLevelGoalProgress(float normalized01, string progressLabel) =>
        ApplyLevelGoalProgressUi(SliderReviveProgress, TextReviveProgress, normalized01);
    public void SetRetryLevelGoalProgress(float normalized01, string progressLabel) =>
        ApplyLevelGoalProgressUi(SliderRetryProgress, TextRetryProgress, normalized01);
    private static void ApplyLevelGoalProgressUi(Slider slider, TextMeshProUGUI text, float
normalized01)
    {
        float n = Mathf.Clamp01(normalized01);
        if (slider != null)
            slider.value = n;
        if (text != null)
            text.text = $"进度{Mathf.RoundToInt(n * 100f)}%";
    }
    private void ClearReviveLevelGoalProgressUi()
    {
        if (SliderReviveProgress != null)
            SliderReviveProgress.value = 0f;
        if (TextReviveProgress != null)
            TextReviveProgress.text = string.Empty;
    }
    private void ClearRetryLevelGoalProgressUi()
    {
        if (SliderRetryProgress != null)
            SliderRetryProgress.value = 0f;
        if (TextRetryProgress != null)
            TextRetryProgress.text = string.Empty;
    }
}

using System;
using FW.Base;
using FW.Unity;
using FW.Base.Procedure;
using Cysharp.Threading.Tasks;
using Events;
using Model.UI;
using Presenter.UI;
using Presenter.Utills;
namespace Presenter
{
    public enum GameId
    {
        None = 0,
        GameXianYu = 1,
        GameMuYu = 2,
        GameDaDiShu = 3,
        GameDaWenZi = 4,
        GameChuiPaoPao = 5,
        GameBlockRelax = 6,
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
public class ProcedureGame : ProcedureBase
{
    private IFSM<IProcedureManager> _fsm;
    private GameId _gameId;
    protected override void OnInit(IFSM<IProcedureManager> fsm)
    {
        Log.Debug("Procedure game init");
        _fsm = fsm;
    }
    protected override void OnEnter(IFSM<IProcedureManager> fsm, object data)
    {
        _gameId = data is GameId id ? id : GameId.None;
        Log.Debug($"Procedure game enter, GameId: {_gameId}");
        Game.Event.Subscribe(GameCommonBarReturnEventArgs.EventId,
OnGameCommonBarReturn);
        Game.Event.Subscribe(TitleLevelUpEventArgs.EventId, OnTitleLevelUp);
        UniTask.Create(DoEnter);
    }
    protected override void OnLeave(IFSM<IProcedureManager> fsm, bool isDestroy)
    {
        Game.Event.Unsubscribe(GameCommonBarReturnEventArgs.EventId,
OnGameCommonBarReturn);
        Game.Event.Unsubscribe(TitleLevelUpEventArgs.EventId, OnTitleLevelUp);
    }
    private void OnTitleLevelUp(object sender, BaseEventArgs e)
    {
        if (e is not TitleLevelUpEventArgs args) return;
        Game.UI.Open<GameAchivementReward>(UINames.GameAchivementReward,
GameAchivementRewardModel.Create(args.NewLevel, args.TitleName)).Forget();
    }
    private void OnGameCommonBarReturn(object sender, BaseEventArgs e)
    {
        Game.UI.Close(UINames.GameCommonBar);
        Game.UI.Close(UINames.CopyText);
        switch (_gameId)
        {
            case GameId.GameXianYu:
                XianYuAudioRuntime.ReleaseBgmOnExit();
                Game.UI.Close(UINames.GameMoYu);
                break;
            case GameId.GameMuYu:
                MuYuBackgroundSwitcher.ReleaseSessionAudioOnExit();
                Game.UI.Close(UINames.GameMuYu);
                break;
            case GameId.GameDaDiShu:
                DaDiShuGame.CleanupOnExit();
                Game.UI.Close(UINames.GameDaDiShu);
                break;
            case GameId.GameDaWenZi:
                DaWenZiWaveSpawner.Instance?.CleanupAllMosquitoes();
                Game.UI.Close(UINames.GameDaWenZi);
                break;
            case GameId.GameChuiPaoPao:

```

方块解压馆软件 V1.0.0 源代码

```
        ChuiPaoPaoGame.CleanupOnExit();
        Game.UI.Close(UINames.GameChuiPaoPao);
        break;
    case GameId.GameBlockRelax:
        BlockRelaxFlowCoordinator.End();
        Game.UI.Close(UINames.GameBlockRelaxEnd);
        Game.UI.Close(UINames.GameBlockRelaxStart);
        Game.UI.Close(UINames.GameBlockRelax);
        break;
    }
    _fsm.ChangeState<ProcedureGameSelect>();
}
private async UniTask DoEnter()
{
    switch (_gameId)
    {
        case GameId.GameXianYu:
            await Game.Scene.LoadSceneAsync("GameMoYu");
            Game.Audio.PlayBGM("BGM_XianYu");
            await Game.UI.Open<GameMoYu>(UINames.GameMoYu, GameMoYuModel.Create());
            break;
        case GameId.GameMuYu:
            await Game.Scene.LoadSceneAsync("GameMuYu");
            if (Game.Audio != null)
                Game.Audio.StopBGM(0f);
            await Game.UI.Open<GameMuYu>(UINames.GameMuYu, GameMuYuModel.Create());
            break;
        case GameId.GameDaDiShu:
            await Game.Scene.LoadSceneAsync("GameDaDiShu");
            await Game.UI.Open<GameDaDiShu>(UINames.GameDaDiShu,
GameDaDiShuModel.Create());
            break;
        case GameId.GameDaWenZi:
            await Game.Scene.LoadSceneAsync("GameDaWenZi");
            await Game.UI.Open<GameDaWenZi>(UINames.GameDaWenZi,
GameDaWenZiModel.Create());
            break;
        case GameId.GameChuiPaoPao:
            await Game.Scene.LoadSceneAsync("GameChuiPaoPao");
            await Game.UI.Open<GameChuiPaoPao>(UINames.GameChuiPaoPao,
GameChuiPaoPaoModel.Create());
            break;
        case GameId.GameBlockRelax:
            await Game.Scene.LoadSceneAsync("GameBlockRelax");
            BlockRelaxFlowCoordinator.Begin();
            await Game.UI.Open<CopyText>(UINames.CopyText, CopyTextModel.Create());
            await Game.UI.Open<GameBlockRelaxStart>(UINames.GameBlockRelaxStart,
GameBlockRelaxStartModel.Create());
            return;
        default:
            Log.Error("Procedure start game select: game id is not found");
            break;
    }
}
```

方块解压馆软件 V1.0.0 源代码

```
        await Game.UI.Open<CopyText>(UINames.CopyText, CopyTextModel.Create());  
    }  
}  
}
```